THYNK
The Hub for Young Next-gen Knowledge

LEVEL 1 · YOUNG EXPLORER · BEGINNERS BASIC 1

My Scratch Adventure

Where Coding meets Vedic Maths — and learning becomes a super-power.

24 SESSIONS | 7 BIG TOPICS | LOADS OF FUN

This workbook belongs to —

MY NAME _____

MY CLASS & SECTION _____

MY SCHOOL _____

MY TEACHER _____

Think Today · Learn Smart · Lead Tomorrow

YOUR MAP FOR THE YEAR

What's Inside My Workbook?

Seven exciting topics that turn you into a real coder — one block, one sutra, one project at a time.

1 Introduction

What is coding? Why are **YOU** here? How will coding + Vedic Maths make you a smarter, faster thinker?

2 Introduction to Scratch

Meet your new playground — sprites, stage, blocks & the magic coding screen.

3 Sequencing & Events

Make things happen in the right order — click, press a key, watch the magic begin.

4 Loops

Repeat and *forever* — paired with the Vedic sutra **Ekadhikena Purvena**.

5 Variables & Lists

Tiny boxes that remember numbers, names and scores for your games.

6 Conditionals (if / else)

Teach the computer to make smart choices — just like you do every day.

7 Operators & Maths Blocks

Add, subtract, multiply — and bring Vedic Maths to life inside your code.

★ Mini-Projects at the End of Every Chapter

Each big topic ends with a fun project — a dancing cat, a counting caterpillar, a number-guessing game and many more — so you don't just *learn* coding, you *build* things.

Exercises & Self-Practice

One full page of fun exercises for every topic — predict, fix the bug, build it yourself. The best way to **really** learn coding is to *do* it on your own!

★ Bonus Capstone · Water Cycle Animation 💧🌞☁️🌧️

The grand finale! Use *everything* you've learned — events, broadcasts, variables, loops **and** cloning — to build a real **Water Cycle** science animation. Take it home, show your family, and watch evaporation, condensation and rain happen on your screen.

How to use this workbook: Read the colourful boxes, try the blocks on screen, finish the activities, and tick the "I did it!" box at the end of every topic. Keep this book safe — by the end of the year it will be full of *your* ideas and *your* code.

CHAPTER ONE

01

Introduction

Welcome, young coder! Before we touch a single block, let's understand why we are about to do something amazing.

WHAT IS CODING? | WHY YOU? | WHY VEDIC MATHS? | WHAT YOU'LL LEARN

1.1 What is Coding?

Big Idea

Coding is the way we **talk to a computer**. We give it small, clear instructions — and the computer does exactly what we say, very, very fast.

DEFINITION A **computer** is a machine that follows orders. It cannot think on its own. **You** are the boss — and the orders you give it are called **code**.

Coding is everywhere around you

You probably use code every day without even knowing it. Look at these everyday things — every single one of them runs on code that someone wrote.

📱 Mobile Apps

YouTube Kids, WhatsApp, your favourite game — all built using code.

🎮 Video Games

Every jump, score and explosion in a game is one tiny piece of code.

🚦 Traffic Signals

Red → Yellow → Green is just code repeating the same steps, day and night.

🚗 Cars

Modern cars use code to brake safely, lock doors and even park themselves.

🤖 Robots

From toy robots to factory machines, all of them follow code we write.

📺 Smart TVs

Picking a Netflix show, changing volume — all powered by tiny lines of code.

DID YOU KNOW?

The very first computer programmer was a girl named **Ada Lovelace**, almost **180 years ago** — long before computers even had screens!

So... how is code different from English?

When you talk to a friend

"Hey, please get me a glass of water if you have time."

Your friend can guess what you mean.

When you talk to a computer

"Walk to kitchen. Open cupboard. Take glass. Walk to filter. Press button. Stop at full."

The computer follows every step exactly.

Coding teaches us to **think clearly, step by step** — and that skill helps in *everything* we do.

1.2 How will Coding help YOU?

You may be thinking — "*I'm only in Grade 4. Why should I learn coding now?*" Great question! Learning to code at your age gives you **super-powers for life**. Here are six big ones.

🧠 Sharper Thinking

Coding is like solving puzzles. Your brain learns to break a BIG problem into tiny, easy steps.

🎨 Wild Creativity

You stop being just a *player* of games — you become a **maker** of games, stories and animations.

🔁 Patience & Practice

Code doesn't always work the first time. You learn how to *try, fix, try again* — without giving up.

🧩 Better at Maths

Loops, numbers, patterns — Scratch *shows* you what maths really looks like.

👫 Teamwork

You'll build projects with friends, share ideas, and discover that two coders are smarter than one.

🚀 Future-Ready

Doctors, designers, pilots, gamers — everyone uses code today. You're getting a head start.

Why now — and not later?

SCIENCE SAYS Your brain at age **9 or 10** is *most* ready to learn new languages — and that includes **coding languages**. The earlier you start, the easier and more natural it feels.

What will be different about you after this year?

1. **You will think like a problem-solver**
   Stuck on homework? You'll break it down the same way you break down a Scratch project.
2. **You will not be afraid of mistakes**
   In coding, mistakes are clues — not failures. They tell you exactly what to fix next.
3. **You will be able to *build* what you imagine**
   Dream of a game where your dog flies? By end of year you'll be able to build it.
4. **You will explain ideas clearly**
   Coding makes you say *exactly* what you mean — a skill teachers and friends will love.

1.3 Why Vedic Maths + Coding?

Big Idea

Vedic Maths teaches your brain to do calculations *super-fast*. **Coding** teaches your brain to think *super-clearly*. Together — you become unstoppable.

WHAT IS VEDIC MATHS? Vedic Maths is a special way of doing arithmetic that comes from **ancient India**. It uses **16 short rules called Sutras** (one-line tricks) to solve big sums in seconds — often without writing them down!

Five reasons Vedic Maths makes you a better coder

⚡ Lightning Mental Maths

You won't depend on a calculator. Your brain becomes the calculator.

🔍 Spotting Patterns

Sutras are pattern recipes. Coding is also full of patterns. Same skill!

🪜 Step-by-Step Thinking

Both Vedic Maths and code break big problems into tiny clean steps.

💪 Confidence in Numbers

When you stop fearing maths, you start *playing* with it — inside your code.

A peek at the Sutras you'll meet this year

Ekadhikena Purvena

"By one more than the previous."

Helps us understand **loops** & counting up.

Nikhilam Navataścaramaṁ

"All from 9 and the last from 10."

Subtracting from 100, 1000 — done in a blink.

Urdhva-Tiryagbhyām

"Vertically and crosswise."

The famous multiplication shortcut.

Why did you join this class?

Whatever your reason, this is what you'll *actually* walk away with:

🎯 The Joy of Making

Building something on a computer that **moves, talks and reacts** — and being able to say "I made this!"

🏆 A Real Project Every Term

By the end of the year, you'll have a portfolio of games and animations to show your family.

1.4 What you'll learn · Why parents smile

By the end of this workbook, you will be able to —

✨ Use Scratch like a pro

- Move and dress sprites
- Make scenes change with backdrops
- Add sounds, music & voices

🧱 Build with code blocks

- Sequence steps in the right order
- Use loops to repeat actions
- Make smart "if-else" choices

🔢 Mix Maths with Code

- Use variables to keep scores
- Add & multiply with operator blocks
- Apply Vedic sutras inside Scratch

🎮 Create your own projects

- A counting game
- An animated short story
- A simple quiz with a score

A note for parents — what THYNK gives your child

🤗 Confidence with Screens

Your child stops being a *consumer* of YouTube and games — and becomes a **creator**.

📚 Stronger School Marks

The logical thinking from coding directly helps in Maths, Science and even English.

🧘 Calmer, Focused Mind

Vedic Maths warm-ups build the kind of focus that helps in exams and life.

🌏 Future-Ready Skills

Coding is the new literacy. Your child will be ready for the world of 2035, not 1995.

🎁 Healthy Screen Time

30–40 minutes a week of *productive* screen time, with something real built at the end.

🤝 Habit of Sharing

Children show their projects at home — a beautiful conversation starter every weekend.

PARENT TIP

Ask your child "*What did you make today?*" instead of "*What did you study today?*" You'll be amazed at what they show you.

CHAPTER TWO

02

Meet Scratch

Your colourful playground where blocks click together like LEGO — and bring sprites to life.

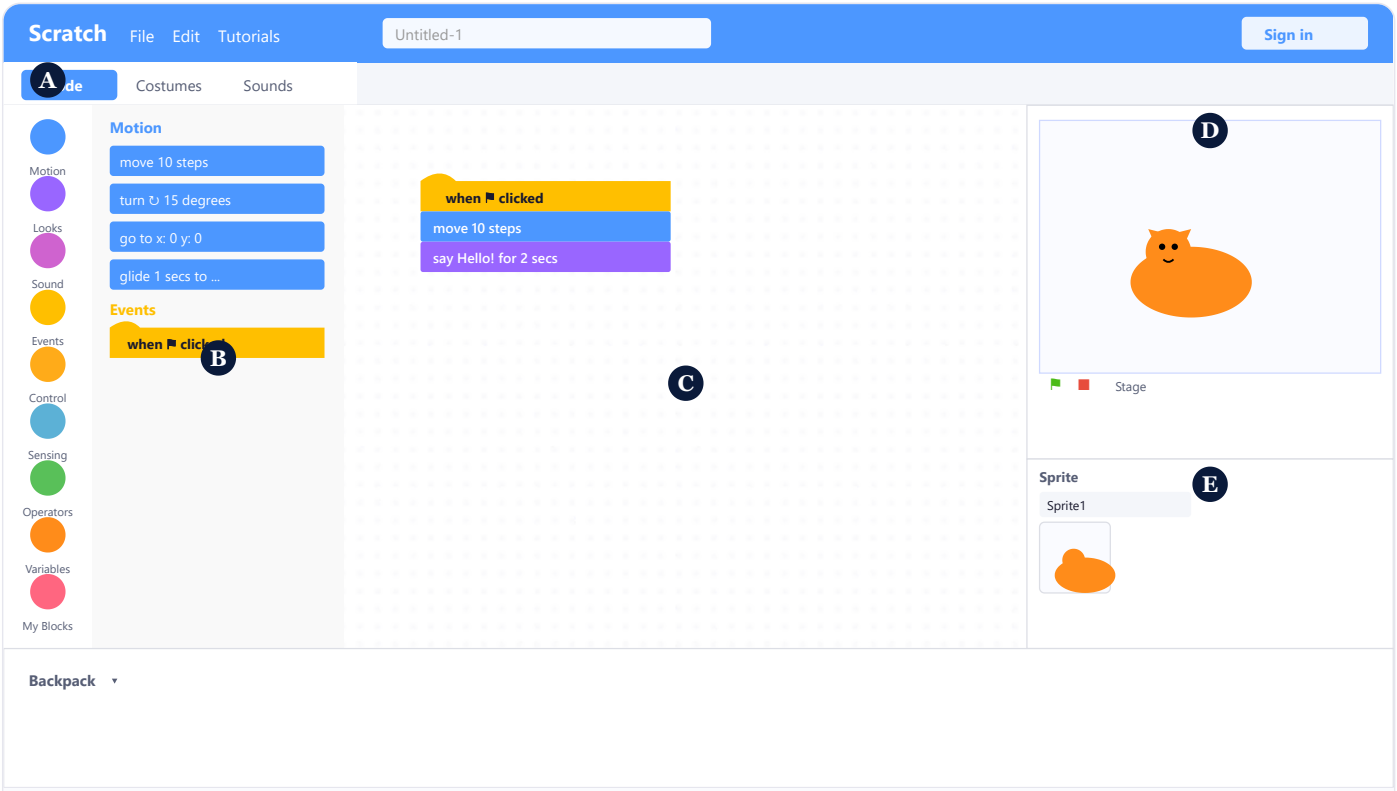
THE STAGE | SPRITES | BLOCK PALETTE | CODE AREA

2.1 The Scratch Screen

Big Idea

Scratch is a **free coding tool** made for children, by MIT. Open **scratch.mit.edu** in any browser and you're ready to code — no installation, no typing tricky words.

Below is a friendly map of the Scratch screen. The five letters **A** to **E** show the five most important areas — learn these and you can find everything!



A simplified picture of the Scratch screen, marked with letters A–E. The real Scratch website looks just like this — open **scratch.mit.edu** and follow along!

A **Block Categories** — coloured circles like *Motion, Looks, Sound, Events*... Click one to see its blocks.

B **Block Palette** — the box of colourful blocks you can drag out and use.

C **Code Area** — your workspace. Drag blocks here and snap them together like LEGO.

D **Stage** — the white screen where your sprites move. Click the green flag ⚑ to start.

E **Sprite List** — every character in your project lives here. Add new ones with the cat-icon button.

2.2 The 9 Colours of Scratch Blocks

Every block in Scratch belongs to a coloured family. Once you remember the colours, you'll know where every block lives!

Motion

Move, turn, glide. Anything to do with **movement**.

move 10 steps

Looks

Say things, change costumes, grow or shrink.

say Hi! for 2 secs

Sound

Play music & sound effects, change the volume.

play sound Meow

Events

"When something happens…" — the **start** of every script.

when ⚑ clicked

Control

Loops & choices — *repeat*, *forever*, *if… then*.

repeat 10

Sensing

Touch, ask, hear — your sprite's senses.

touching edge ?

Operators

Maths & comparisons — add, subtract, < >.

5 + 3

Variables

Boxes that remember numbers, scores & names.

change score by 1

My Blocks

Make your *own* blocks — your private super-powers!

jump high

Three tabs above the block palette

📜 Code

The default tab. This is where you build your scripts.

👕 Costumes

Change how your sprite *looks* — draw, colour, even add a moustache.

🎵 Sounds

Add or record sounds — meows, claps, your own voice.

TRY IT! · 5 MINUTES

Hello, Scratch!

1. Open **scratch.mit.edu** and click **Create**.
2. Find the yellow **Events** category. Drag **when ⚑ clicked** into the Code Area.
3. From purple **Looks**, drag **say Hello! for 2 seconds** and snap it under the yellow block.
4. Click the green flag ⚑ at the top of the Stage. Did the cat speak? 🎉

Meet Your A.I. Buddy

Stuck on a block? Just ask!

From here on, every chapter has a purple **"Ask A.I."** box. It shows you exactly what to type into a chat A.I. (like ChatGPT, Gemini or Copilot) when you get stuck — so you never feel lost.

What is A.I., really?

A.I. (Artificial Intelligence) is a **computer friend** that has read millions of books, lessons and Scratch projects. It can explain ideas in your own words, suggest blocks you might have forgotten, and help you find tiny mistakes — but **only if you know how to ask it nicely**. Think of it like a super-patient tutor who never gets tired.

3 Golden Rules of Asking A.I.

1

Be Clear

Say **what you want** the sprite to do — not just "fix it". Example: "*I want my cat to spin twice when I press space.*"

2

Be Specific

Tell A.I. **which blocks you tried** and **what went wrong**. The more details, the better the help.

3

One Step at a Time

Ask about **one problem per chat**. Don't pile three questions together — A.I. helps best when focused.

COMPARE THESE TWO PROMPTS

```
â□Œ my code doesnt work fix it
```

```
âœ… In Scratch, I want my cat sprite to say "Hi!" for 2 seconds and then move 100 steps when I press the green flag. I tried using "say Hi" and "move 100" but the cat moves first and then says Hi. How do I fix the order?
```

See the difference? The second prompt tells A.I. **what** you want, **what you tried**, and **what's wrong** — so it can give a real answer.

The Golden Rule — Always **try yourself first**. Drag the blocks, watch what happens, think for a minute. Only ask A.I. when you've genuinely tried and got stuck. A.I. is your *helper*, not your homework-doer. The more you struggle a little, the smarter **you** get. 🧠✨

CHAPTER THREE

03

Sequencing & Events

First, second, third… coding is all about doing things in the right order — and starting them at the right moment.

SEQUENCE | WHEN ⚑ CLICKED | KEY PRESS | FIRST SCRIPT!

3.1 What is a Sequence?

Big Idea

A **sequence** is a list of steps done **one after the other**, in a fixed order. Every program ever written is just a smart sequence of small steps.

Your morning is a sequence

Think about how you get ready for school. Without even thinking, you follow a sequence:

1. **Wake up**
   Alarm rings → you open your eyes.

2. **Brush your teeth**
   Open tap → squeeze toothpaste → brush → rinse.

3. **Wear your uniform**
   Shirt first, **then** tie. Not the other way around!

4. **Eat breakfast & leave**
   Eat → wear shoes → pick bag → walk to bus.

WHAT HAPPENS IF ORDER CHANGES?

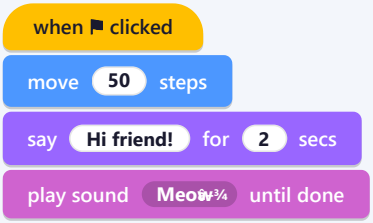
Imagine wearing your **shoes before** your socks, or eating breakfast *after* brushing your teeth. Funny — but it doesn't work, right? **Order matters**. The same is true in code.

In Scratch, blocks *snap* in sequence

When you stack Scratch blocks **from top to bottom**, they run in that exact order. The top block runs first, then the next, then the next.

🧱 Block stack

CODE AREA · TOP → BOTTOM



🗣 In simple words

1. Wait till someone clicks the green flag.
2. **First**, walk 50 steps forward.
3. **Then** say "Hi friend!" for 2 seconds.
4. **Finally**, play the meow sound.

REMEMBER Swap any two blocks and the result **changes**. Coders must be careful planners!

3.2 Vedic Warm-up · Number Sequences

Big Idea · Find the Secret Rule

A **number sequence** is a row of numbers that follows a **secret rule** — like "*add 2 each time*" or "*minus 5 each time*". Spot the rule and you can predict any missing number — that is exactly what a computer loop does!

HOW TO SPOT THE RULE — IN 3 EASY STEPS **1.** Take any two neighbours — ask "*what did we add or subtract?*" **2.** Check the same jump on a second pair. **3.** If the jump is the **same every time**, that is your rule — apply it to fill any blank.

*Example — **3, 6, 9, 12, __** : 6−3=3, 9−6=3 ⇒ rule = **add 3**, next = **15**. Try minus too: **20, 17, 14, __, 8** ⇒ rule = **minus 3**, missing = **11**.*

Your turn · Circle the correct answer

1 **Rule of** `2, 4, 6, 8, 10, 12, 14`
a. Add 1 b. Add 2

2 **Next in** `4, 8, 12, ___`
a. 10 b. 16 c. 18

3 **Rule of** `18, 15, 12, 9, 6, 3, 0`
a. Add 3 b. Plus 4 c. Minus 3 d. Minus 4

4 **Missing in** `12, ___, 32, 42, 52`
a. 22 b. 23 c. 24

5 **Rule of** `103, 107, 111, 115, 119, 123`
a. Add 100 b. Add 104 c. Add 4 d. Minus 4

6 **Noora** starts at 13, subtracts 4 each time. Next 3 numbers?
a. 14, 17, 20 b. 9, 5, 1 c. 12, 11, 10 d. 9, 4, 0

7 **Next in** `150, 155, 160, 165, ___`
a. 170 b. 175 c. 180

8 **Rule of** `100, 200, 300, 400, 500`
a. Add 100 b. Add 10 c. Minus 10

9 **Missing in** `58, 51, 44, ___, 30`
a. 42 b. 37 c. 35

10 **Next in** `180, 140, 100, 60, ___`
a. 40 b. 50 c. 30 d. 20

Figure out the pattern · Write the missing number

11 `12, 15, ___`
Answer: _____

12 `4, 8, 12, 16, 20, ___`
Answer: _____

13 `___, 10, 15, 20, 25`
Answer: _____

14 `2, 4, 6, 8, 10, ___`
Answer: _____

BRIDGE TO SCRATCH

Every rule you spotted — "*add 2*", "*minus 3*" — is the recipe for a Scratch loop with a counter: **change counter by 2**, **change counter by −3**. Master the pattern here (and a couple more practice sheets coming up!) and the **Events** section will feel natural.

10 quick pattern puzzles — find the rule, pick the answer!

Look at two neighbours, spot the jump, check it on another pair — then circle the right option. Total questions: **10**. Suggested time: **15–20 mins**.

1 **Next in** `9, 18, 27, 36, 45, ___`

a. 54 b. 76
c. 63 d. 81

2 `6, 9, 12, 15, 18, 21` — what is the rule?

a. × 6 b. + 3
c. × 4 d. × 5

3 **Missing blanks in** `69, 60, 51, ___, 33, ___`

a. 42, 24 b. 45, 21
c. 40, 22 d. I don't know

4 **Rule of** `65, 62, 59, 56, 53, 50`

a. Subtract 2 b. Subtract 3
c. Subtract 4 d. Add 3

5 **Next in** `11, 9, 7, 5, ___`

a. 7 b. 4
c. 3 d. 2

6 **Magic Box** — what is the rule?

Column A	Column B
2	6
3	9
4	12
5	?
6	?
7	?

a. Add 4 b. Add 6
c. Multiply by 2 d. Multiply by 3

7 **Next in** `55, 60, 65, ___, 75, 80`

a. 70 b. 66
c. 40 d. 75

8 **Rule of** `99, 97, 95, 93, 91`

a. Subtract 1 b. Add 2
c. Subtract 2 d. Add 3

9 **Missing in** `23, 33, ___, 53, ___, 73`

a. 32, 62 b. 42, 63
c. 43, 63 d. 42, 62

10 **Rule of** `32, 36, 40, 44`

a. add 4 b. multiply 2
c. subtract 4 d. divide by 2

Tip — If neighbours **grow**, the rule is *add* or *multiply*. If they **shrink**, it's *subtract* or *divide*. Always check your rule on **two pairs** before you commit!

12 more pattern puzzles — numbers *and* shapes!

A mix of number sequences, dot & block patterns, and repeating shapes. Total questions: **12**. Suggested time: **25–35 mins**.

1 **Rule of** `5, 10, 15, 20, 25`

a. add 6
b. add 5
c. add 4
d. add 7

2 **Missing in** `18, 24, ___, 36, 42`

a. 28
b. 31
c. 30
d. 35

3 **Missing in** `22, 29, 36, ___, 50, ___`

a. 35, 55
b. 40, 57
c. 41, 56
d. 43, 57

4 **How many counters come next?**

a. 5
b. 7
c. 9
d. 11

5 **Next letter:** `A B C A B C A B C ___`

a. A
b. B
c. C

6 **How many blocks in the 4th term?**

a. 16 blocks
b. 15 blocks
c. 14 blocks
d. End of pattern

7 **Missing in** `69, 60, 51, ___, 33, ___`

a. 42, 24
b. 45, 21
c. 40, 22
d. I don't know

8 **Missing in** `120, 115, 110, ___, 100, 95`

a. 105
b. 115
c. 90
d. I don't know

9 **Missing in** `230, 219, 208, 197, ___, 175`

a. 187
b. 186
c. 195
d. I don't know

10 **Start with 1, multiply the previous by 3.** `1, ___, ___, ___, ___`

a. 1, 3, 6, 9, 12
b. 1, 4, 7, 10, 13
c. 1, 3, 9, 12, 15
d. 1, 3, 9, 27, 81

11 **11th figure in this pattern?**

a. Diamond
b. Hexagon
c. Trapezoid

12 **Which pattern follows this rule?** *Start with 2, add 4 each time until there are 5 numbers.*

a. 2, 4, 8, 12, 16
b. 2, 6, 10, 14, 17
c. 2, 6, 10, 14, 18
d. 2, 6, 11, 15, 19

Tip — For a **shape** or **letter** pattern like *A B C A B C…*, count how many items repeat (the **cycle**) and then divide the position by that cycle — the remainder tells you which shape/letter you'll see!

16 quick puzzles — hop, skip & back-count!

Count in 10s, 20s, 50s, 100s and 1,000s — and use **1 more** / **1 less**. Total: **16 Qs** · Suggested time: **16 mins**.

1 **Next in the shape pattern:**

a. Purple right-tri b. Blue triangle
c. Yellow triangle d. Red right-tri

2 **Missing in 20, 30, ___, 50, 60**

a. 30 b. 50
c. 40 d. 04

3 **Next in 320, 340, 360, ___**

a. 54 b. 400
c. 380 d. 370

4 **Missing in 150, 200, ___, 300**

a. 250 b. 225
c. 275 d. 350

5 **Count back by 10s: 90, ___, ___, ___**

a. 80, 70, 60 b. 80, 70, 50
c. 100, 110, 120

6 **Missing in 520, ___, 560, 580, ___**

a. 540, 590 b. 540, 600
c. 525, 590 d. 510, 590

7 **Missing in 600, 550, ___, ___, 400, 350, 300, 250, 200**

a. 500, 400 b. 350, 500
c. 450, 500 d. 500, 450

8 **Missing in 1,403, ___, 3,403, 4,403**

a. 1,403 b. 2,000
c. 2,503 d. 2,403

9 **Next number: 1,680, 1,780, 1,880, ___**

Answer: _____

10 **Next number: 6,357, 6,358, 6,359, ___**

Answer: _____

11 **5,447 is 1,000 more than 4,447.**

a. True b. False

12 **1 more than 8,902 is __**

a. 8,901 b. 8,903
c. 8,900

13 **1 less than 3,450 is __**

a. 3,449 b. 3,451
c. 3,452

14 **10 less than 2,475 is 2,485.**

a. True b. False

15 **__ is 10 less than 650.**

a. 670 b. 660
c. 640

16 **6,046 is __ less than 7,046.**

a. 1,000 b. 100
c. 10

Tip — For **"1 more" / "1 less"**, only the last digit changes. For **"10 more" / "10 less"**, only the tens digit changes. For **"100 more"**, only hundreds. Line up the place values in your head!

10 Input & Output puzzles — find the rule!

A **rule** tells the machine how to turn every **input** into an **output**. Spot it once, and every row falls into place. Total: **10 Qs** · Suggested time: **13 mins**.

1 **What real-life thing do we use as a picture for the input & output table?**

a. A soda machine
b. A television
c. Shoes
d. A transformer

2 **What never changes?**

a. The rule
b. The output
c. The input
d. The pattern

3 **What is the output?**

◇	♥
10	15
15	20
20	25
25	?

a. 35
b. 40
c. 30
d. 50

4 **What is the output?**

◇	♥
900	800
700	600
500	?
300	200

a. 300
b. 600
c. 100
d. 400

5 **Correct values for the outputs?** *(pick two)*

◇	♥
10	21
12	23
13	?
14	?

a. 22
b. 24
c. 10
d. 25

6 **Correct values for the outputs?** *(pick two)*

◇	♥
550	50
450	400
350	?
150	?

a. 300
b. 250
c. 200
d. 100

7 **Rule (number expression)?**

★	Rule	☺
17	?	24
18	?	25
27	?	34
45	?	52

a. +8
b. ×6
c. +7
d. ×5

8 **Fill in the missing input & output.** *(pick two)*

★	☺
1	9
3	11
5	13
?	15
9	?

a. Input 6
b. Input 7
c. Output 17
d. Output 16

9 **Number Machine rule (input → output)?**

Input	Output
1	10
2	11
3	12
4	13

a. −9
b. ×10
c. ×4
d. +9

10 **Position → Value — which rule?**

Position	Value
1	33
2	34
3	35
4	36

a. ×33
b. −32
c. ÷33
d. +32

Tip — An **Input/Output table** is a mini computer! Check the rule on **two** rows before you trust it — the **same rule must work for every row**. That's exactly how Scratch's **green Operator blocks** will behave in Chapter 7.

3.3 What is an Event?

DEFINITION An **event** is something that **happens** — like a click, a key being pressed, or a message being sent. Events are the **starting whistle** for our sequences.

Yellow "Hat" blocks live in the Events family

Event blocks are shaped like a *hat* — round on top. They **sit at the top** of every script. Nothing snaps above them, because nothing comes *before* the start!

🚩 Green Flag

when ⚑ clicked

Runs when you click the green flag above the stage.

⌨️ Key Press

when space key pressed

Runs the moment you press the chosen key.

🐱 Sprite Click

when this sprite clicked

Runs when you click *this* sprite on the stage.

Sample Project · "Greeting Cat" 🐱

Goal: When you click the green flag, the cat walks and greets you. When you press **space**, it dances by spinning.

Script 1 · Greeting

WHEN THE GREEN FLAG IS CLICKED

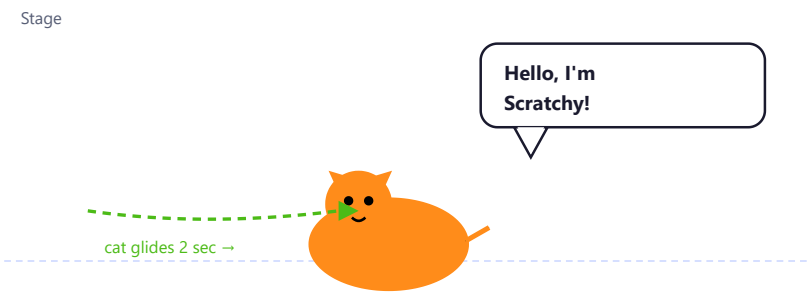
```
when ⚑ clicked
go to x: -150 y: 0
glide 2 secs to x: 0 y: 0
say Hello, I'm Scratchy! for 2 secs
```

Script 2 · Dance

WHEN THE SPACE KEY IS PRESSED

```
when space key pressed
turn ↻ 360 degrees
play sound Meow until done
```

What you'll see on the Stage 🎬



After clicking ⚑ — the cat glides to the centre and greets you. Press **space** and watch it spin!

YOUR TURN · ACTIVITY 3.A

Make Scratchy reply with your name

1. Open the Greeting Cat project above.
2. Change the speech text from "Scratchy" to your own name.
3. Add a third script that runs **when this sprite clicked** and makes the cat say "Ouch!" for 1 second.
4. ☐ I did it!

ðŸ¤–

ASK A.I. **Stuck? Try this prompt —**

```
âœï¸ I'm coding in Scratch. I want my cat sprite to say "Hi, friend!" for 2 seconds and then
move 80 steps when I click the green flag. I tried two blocks but the cat moves before it speaks.
Which Events block should I start with, and what's the correct order of blocks so it talks first,
then walks?
```

💡 *Notice how the prompt names the **topic** (sequencing & events) and tells A.I. **what went wrong**.*

Get the order right!

Three challenges to test your sequencing super-power. Pencil ready — let's go.

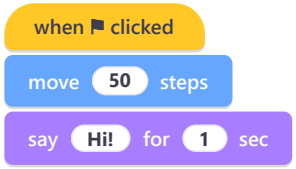
1 PREDICT What does the cat *do*?

Read this stack from top to bottom. In which order will things happen on the stage? Write 1, 2, 3, 4 in the boxes on the right.

Order	What happens
	Cat turns 15°
	Cat says "Hello!"
	Cat moves 30 steps
	Cat plays meow sound

2 FIX THE BUG Wrong order!

Riya wants her cat to **say "Hi"** first, **then jump** (move 50 steps). But she wrote the blocks the wrong way round. **Cross out** the wrong stack and **rewrite the correct one** in the empty box.



3 BUILD IT! The Birthday Sprite 🎂

Build this in Scratch:

- When the **space key** is pressed, the cat should say **"Happy Birthday!"** for 2 seconds.
- Then it plays a **sound** of your choice.
- Then it says **your friend's name** for 2 seconds.

Which **Events** block will start your script? Write its name:

_____

☐ I built and tested my Birthday Sprite!

Tip — The block at the **top** always runs first. The order of the rest *changes the story*.

CHAPTER FOUR

04

Loops

Why write the same step 100 times when you can tell the computer "do this 100 times" just once?

REPEAT FOREVER EKADHIKENA PURVENA DRAW A SQUARE

4.1 Why do we need Loops?

Big Idea

A **loop** tells the computer to **repeat** a set of steps. Loops save time, save typing, and let one tiny piece of code do BIG things.

Loops in real life

Brushing

Move brush up-down, up-down, up-down... 40 times. That's a loop!

Climbing stairs

Lift foot → step up → repeat for every step till you reach the top.

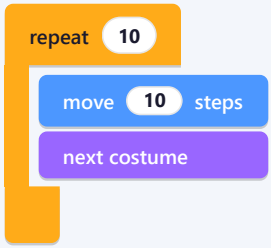
Breathing

Breathe in, breathe out — **forever**. Your body never stops looping.

Two famous loop blocks in Scratch

repeat (a fixed number of times)

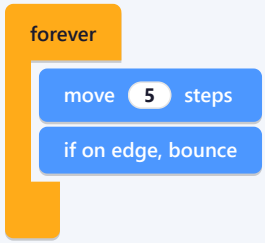
REPEATS INSIDE BLOCKS 10 TIMES, THEN STOPS



Use this when you know **exactly how many** times you want something to happen.

forever (never stops)

KEEPS RUNNING UNTIL YOU PRESS THE RED STOP BUTTON



Use this for things that should **keep going** — like a sprite that always follows the mouse.

The Magic Bridge · Vedic Sutra meets Code

VEDIC SUTRA

Ekadhikena Purvena

"By one more than the previous."

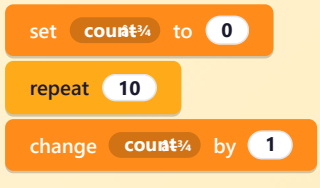
Counting **1, 2, 3, 4...** is nothing but starting at 1 and adding **one more** every time. That is exactly what a loop does in code.

→

IN SCRATCH

repeat & counter

Each time the loop runs, our counter becomes **one more than the previous** — the sutra in action!



4.2 Vedic Warm-up · Ekadhiken

Big Idea · "One more than the previous"

The Vedic sutra **Ekādhikena Pūrveṇa** means ***"by one more than the previous"***. When we mark a digit with a little star ★, the rule is simple: **add 1 to that digit only**. Every other digit stays exactly the same.

RULE OF THE STAR ★ **Ekadhik = add 1 to the marked digit.** This tiny rule powers many Vedic maths tricks — squaring numbers ending in 5, base-10 arithmetic, and (as we'll soon see) the way loops **count up** inside a computer.

How to solve — in 3 easy steps

1 Spot the star ★

Read the question — e.g. "*Of 6 in 65*" — and find the digit it points to. Draw a tiny star above it: **6★5**.

2 Add 1 to it

Only the marked digit changes. **6 → 7**, **8 → 9**, **2 → 3**. Everything else is untouched.

3 Rewrite the number

Put the new digit back in the same place. Copy the other digits exactly. That's your answer!

Worked examples

Ekadhiken · walk-through — *Ekādhikena Pūrveṇa · "by one more than the previous"*

OF 4 IN 4	Only one digit — mark it: **4★**. Add 1 → **5**
OF 8 IN 8	Only one digit — mark it: **8★**. Add 1 → **9**
OF 2 IN 42	Mark the 2: **4 2★**. The **4** stays; the **2** becomes **3** → **43**

Your turn · Practice sheet

A ★ next to a digit means "*this is the marked one*". Add 1 to that digit, keep the rest the same, and write your answer in the last column.

SR.	QUESTION	EKADHIK ANSWER
1	Of 6 in 65	
2	Of 5 in 57	
3	Of 9 in 91	
4	Of 3 in 73	
5	Of 7 in 127	
6	Of 8 in 286	

SR.	QUESTION	EKADHIK ANSWER
7	Of 1 in 101	
8	Of 5 in 35	
9	Of 4 in 248	
10	Of 6 in 462	
11	Of 2 in 823	
12	Of 9 in 192	

BRIDGE TO SCRATCH

Every time a Scratch **loop** runs, its counter jumps **one more than before** — *1 → 2 → 3 → 4* … That is Ekadhiken at work inside a computer! Nail this warm-up and the **Draw a Square** project coming up will feel like a game you already know.

4.3 Vedic Warm-up · Ekadhiken Poorven

Big Idea · "One more than the *previous*"

Poorven (also spelt *Pūrva / Purvena*) is the Sanskrit word for **"previous"** — the digit that sits **just to the left** of the one we're looking at. So **Ekadhiken Poorven** means: **add 1 to the digit that comes *before* the marked digit**. The marked digit itself does not change!

RULE OF THE LEFT-NEIGHBOUR **Ekadhiken Poorven = add 1 to the digit on the LEFT of the marked digit.** The marked digit and all the others stay exactly the same. *Different* from plain Ekadhiken — here it's the **left neighbour** that gains a 1, not the marked digit.

How to solve — in 3 easy steps

1 Star the marked digit ★

Read the question — e.g. "*Poorven of 7 in 378*" — and mark the digit it names: **3 7★ 8**.

2 Look one step to the LEFT ←

Find the digit sitting **just before** the star. That is the **poorven** (previous) digit. In **3 7★ 8**, the previous is **3**.

3 Add 1 to *that* digit

Bump the previous digit up by 1. Keep every other digit — including the marked one — exactly as it was. **3 → 4**, so **378** becomes **478**.

Worked examples

Ekadhiken Poorven · walk-through — *Poorven · "of the previous"*

POORVEN OF 7 IN 378	Mark the 7: **3 7★ 8**. The previous (left) is **3**. 3+1=4 → **478**
POORVEN OF 8 IN 684	Mark the 8: **6 8★ 4**. The previous is **6**. 6+1=7 → **784**
POORVEN OF 9 IN 590	Mark the 9: **5 9★ 0**. The previous is **5**. 5+1=6 → **690**

Your turn · Practice sheet

Star the marked digit, look one step to its **LEFT**, and add 1 to that neighbour. Write the new number in the last column.

*Tip: if the marked digit has **no** neighbour on the left, imagine a tiny **0** sitting there — then add 1 to that 0.*

SR.	QUESTION	EKADHIK ANSWER
1	Poorven of 5 in 54	
2	Poorven of 7 in 378	
3	Poorven of 8 in 684	
4	Poorven of 3 in 931	
5	Poorven of 6 in 4628	

SR.	QUESTION	EKADHIK ANSWER
8	Poorven of 4 in 947	
9	Poorven of 1 in 213	
10	Poorven of 7 in 1705	
11	Poorven of 5 in 451	
12	Poorven of 8 in 9876	

SR.	QUESTION	EKADHIK	ANSWER
6	Poorven of 9 in 590		
7	Poorven of 2 in 120		

SR.	QUESTION	EKADHIK	ANSWER
13	Poorven of 3 in 330		
14	Poorven of 6 in 961		
15	Poorven of 2 in 2045		

EKADHIKEN VS. EKADHIKEN POORVEN

Ekadhiken adds 1 to the **marked** digit itself. **Ekadhiken Poorven** adds 1 to the digit **just before** it. Same "+1", different target! You'll see the **Poorven** rule return in Chapter 7 when we **square any number ending in 5** — that trick is Ekadhiken Poorven in disguise.

4.4 Mini-Project · Draw a Square

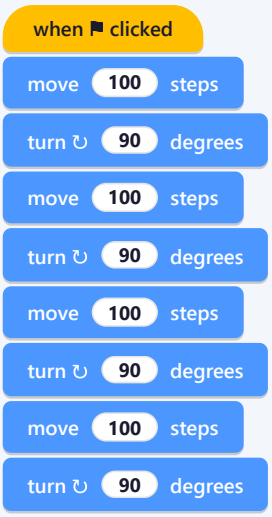
A square has **4 equal sides** and **4 right angles**. To draw it, our sprite must do the same two steps four times: **move** forward, then **turn** 90°. Perfect job for a loop!

Step 1 · Set up the pen

Scratch has a built-in **Pen** extension that lets a sprite draw on the stage. Click the blue **"Add Extension"** button at the bottom-left of the screen and choose **Pen**. New dark-green blocks will appear!

Without a loop 😩

8 BLOCKS. SO MUCH TYPING!

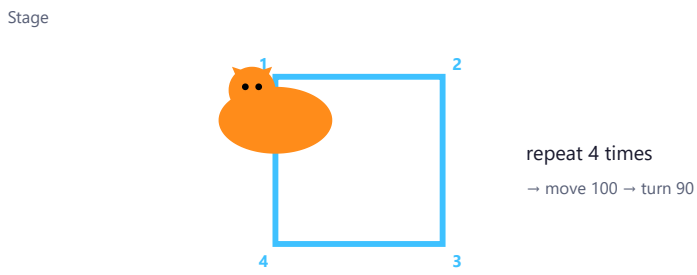


With a loop 😻

JUST 4 BLOCKS — SAME RESULT!

"pen down" is from the Pen extension — it acts like a coloured marker stuck to the sprite.

What you'll see on the Stage 🎬



The sprite draws all four sides of a perfect square, one corner after another.

Think like a coder 🧠

What if we change repeat to 3?

You will get a **triangle**... *almost*! The turn must change to **120°**, because 360 ÷ 3 = 120.

What if we change repeat to 6?

You'll draw a **hexagon** — use a **60°** turn. (360 ÷ 6 = 60). See the pattern?

VEDIC CONNECT

One more than the previous

From triangle (3 sides) → square (4) → pentagon (5) → hexagon (6)... every new shape is **Ekadhikena Purvena** — one more side than the previous!

TRY THIS

Shape factory

→ Change **repeat** `N` and turn by **360 ÷ N** degrees. Make a new shape every minute and stick them in your workbook!

YOUR TURN · ACTIVITY 4.A

Build the Shape Factory

1. Make the cat draw a **square** (repeat 4, turn 90).
2. Stop, change to repeat 3 + turn 120 — get a **triangle**.
3. Now try repeat 6 + turn 60 — get a **hexagon**.
4. Bonus 🌟: repeat 36 + turn 10 — what shape did you get? Draw your guess below:

ðŸ¤–

ASK A.I. **Stuck? Try this prompt —**

> âœï¸ In Scratch, my cat draws a **square** using **repeat 4 → move 100 → turn 90°** with the Pen extension. I want it to draw a **hexagon (6 sides)** instead. What number should I put inside **repeat ___** and **turn ___ degrees**? Please share the formula so I can try other shapes too (octagon, decagon).

💡 *Asking for the **formula** is smarter than asking for one answer — you learn the rule, not just the result.*

Repeat & Forever — your turn!

Save your fingers from doing the same thing 100 times. Let the loop do it for you.

1 COUNT IT How many times?

Each row shows a loop block. Write how many times the inside blocks will run.

Loop	Inside runs how many times?
repeat 10	
repeat 25	
forever	_____ until you press ■
repeat 4 inside repeat 3 (loops *inside* loops)	

2 MATCH THE SHAPE repeat × turn = ?

Fill in the missing pieces. **Rule:** turn = 360 ÷ number of sides.

Shape	repeat	turn (degrees)
Triangle	3	
Square		90°
Pentagon	5	
Octagon		

3 BUILD IT! Rocket Countdown 🚀

Build a script that counts down from **10 to 1**, and then says **"BLAST OFF!"** for 2 seconds.

1. Make a variable called **counter** and set it to 10.
2. Use a **repeat 10** loop — inside it, the cat says **counter**, waits 1 sec, then **changes counter by −1**.
3. After the loop, the cat shouts **"BLAST OFF!"**

Which value should you put inside **change counter by** ___ ?

_____

☐ My rocket counted down successfully!

Tip — A loop is just "*do this again*". If you find yourself copy-pasting blocks more than twice — use a loop.

CHAPTER FIVE

05
Variables & Lists

Tiny labelled boxes inside the computer's brain — they remember your scores, names and answers so your code can do magic with them.

VARIABLES | SET / CHANGE | LISTS | VEDIC QUIZ

5.1 What is a Variable?

Big Idea

A **variable** is a **labelled box** in the computer's memory. We give it a *name*, and we can put a *number* or a *word* inside. Later we can take it out, change it, or use it in our code.

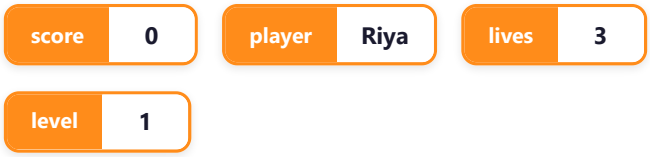
Think of variables like school lockers

Every locker has —

- A **name plate** ("Riya", "Aarav") so you know whose it is.
- An **inside** where you store something (a book, a tiffin).
- A **door** you can open to *change* what's inside.

A variable is exactly the same — only the locker is invisible and lives *inside* the computer.

My Scratch lockers right now —



*The orange tag is the variable's **name**; the white box is the **value** inside.*

How to make a variable in Scratch

1 **Click the orange Variables category**

It's the second-last category in the block palette.

2 **Click "Make a Variable"**

A small window pops up.

3 **Type a name**

Use a name that *describes* what's inside — like **score**, not **x**. Coders are clear, not lazy!

4 **Click OK**

Boom! New blocks appear and a tiny orange box pops up on the stage showing your variable.

The 4 super-power blocks of variables

Set

Put a *fresh* value into the box (throws away whatever was there before).

Change

Add a number to whatever is already inside.

Show

Make the variable's box visible on the stage.

Hide

Hide the box from the stage (it still keeps the value!).

SET VS. CHANGE — CAREFUL!

set score to 5 means "make score equal to 5". **change score by 5** means "add 5 to whatever score already is". Mixing them up is the #1 reason quiz scores look wrong!

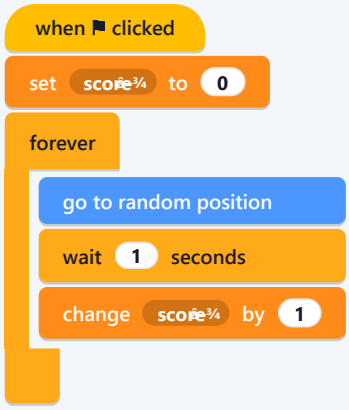
5.2 Variables in Action · A Tiny Score

Let's see how variables actually *change* while a program runs. Every line in the code below tells us what the variable looks like **at that moment**.

Walk-through · "Catch the Star" 🌟

The script

SPRITE: STAR · RUNS WHEN ⚑ CLICKED



What's inside *score* ?

Time	Block running	score
0 sec	flag clicked	score —
just after	set score to 0	score 0
1 sec	change by 1	score 1
2 sec	change by 1	score 2
3 sec	change by 1	score 3
…	forever	score ∞

Vedic Connect — variables are *Dhāraṇā* (मेमरी)

VEDIC IDEA

Hold the number in your head

In mental maths, you often **hold a partial answer** in your head while doing the next step (e.g. "2+3=5… now add 7… now 12"). That holding place is a variable!

→

IN SCRATCH

The variable IS your scratchpad

Use a variable called **running_total** or **answer** to remember the in-between number. Your sprite never forgets.

YOUR TURN · ACTIVITY 5.A

Coin Counter

1. Make a variable called **coins**.
2. Write a script: **when ⚑ clicked → set coins to 0**.
3. Add another script: **when this sprite clicked → change coins by 1**.
4. Click your sprite 5 times. The coins box on the stage should jump from 0 → 5. ✅

☐ I made my first counter!

5.3 What is a List?

Big Idea

A variable holds **one** thing. A **list** is many variables in a neat row — each item has a number called its **index**, starting from **1**.

Lists you already use every day

🛒 Shopping list

Bread, milk, eggs, apples — items listed in order so nothing is forgotten.

🗓️ Class timetable

Period 1, Period 2, Period 3… each slot has a subject.

🏆 Top scorers

1st, 2nd, 3rd place — exactly like a list of scores in a game.

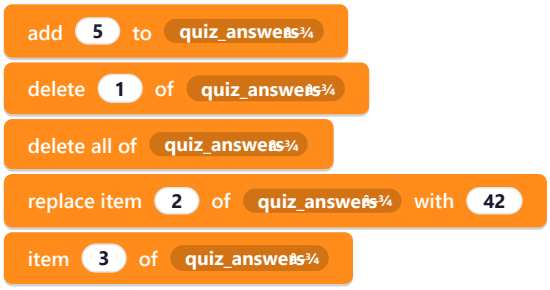
A list looks like this on the stage

List name: *quiz_answers*

quiz_answers	length 5
1	99
2	88
3	9801
4	12
5	144
+	

Position 1 holds 99, position 2 holds 88, and so on.

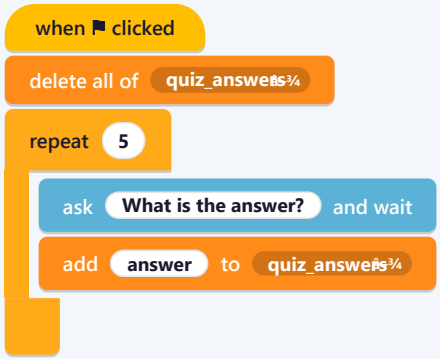
The 5 most useful list blocks



Use **add** when collecting answers as the player plays. Use **item N of …** to read an answer back later.

Building a list as the user plays

A LIST GROWS AUTOMATICALLY AS THE PLAYER ANSWERS EACH QUESTION



INDEX STARTS AT 1

Most professional coding languages start counting at 0 — but **Scratch is friendly** and starts at 1, just like a class register.

5.4 Mini-Project · Vedic Quiz · Score Keeper

What we're building

A 5-question mental-maths quiz. The cat **asks** a question, the player **types** the answer, and Scratch keeps the **score** in a variable + saves every reply in a list. (We'll add the "Right or Wrong" check in the next chapter — for now, just the structure!)

Variables & List we'll need

score (variable)

Starts at 0. Goes up by 1 each correct answer.

q_no (variable)

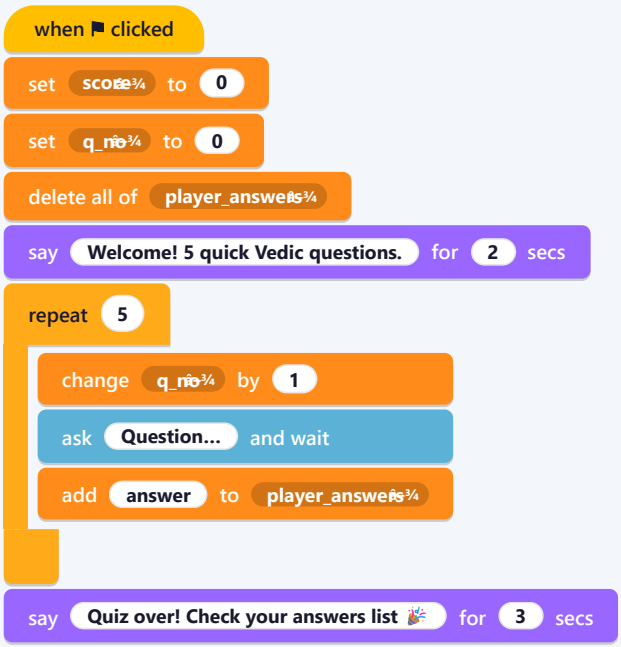
Tells us which question we're on (1, 2, 3, 4, 5).

player_answers (list)

Stores every answer the player typed.

The complete script

SPRITE: MATHS-CAT · RUNS WHEN ⚑ CLICKED



After the quiz, the stage looks like —

score	0

q_no	5

Score is still 0 because we haven't checked answers yet (that's Chapter 6's job). But **q_no** proudly shows we asked all 5 questions.

player_answers	length 5
1	99
2	88
3	9801
4	12
5	144

YOUR TURN · ACTIVITY 5.B

Type your 5 Vedic questions

Plan your quiz *before* coding. Write 5 fun mental-maths questions you'd like the cat to ask:

1. Q1: .....
2. Q2: .....
3. Q3: .....
4. Q4: .....
5. Q5: .....

☐ I built the quiz skeleton!

ðŸ¤– ASK A.I. **Stuck? Try this prompt —**

```
âœï¸ I'm building a Scratch quiz with a variable called score and a list called player_answers. After each question I want to add the player's answer to the list and show the score on the stage. But every time the cat asks the next question, my score box shows 0. I'm using set score to 0 at the start. What might be making the score reset, and how do I fix it?
```

💡 *Telling A.I. the **variable names** you used (score, player_answers) helps it explain in the same language as your code.*

Boxes that remember!

Track values, fix bugs and build your own little memory machine.

1 TRACE IT What's in the score box?

Start: **score = 0**. Fill the table after each instruction.

Instruction	score is now...
change score by 5	
change score by 3	
set score to 10	
change score by −4	
change score by 1	

2 LIST TRACE What's in the list?

Start with an **empty** list called **fruits**. Apply the steps in order, then write the final list:

1. add **"apple"** to fruits
2. add **"banana"** to fruits
3. add **"cherry"** to fruits
4. delete item **1** of fruits
5. add **"date"** to fruits

Final fruits list (in order):

1 _____ 2 _____ 3 _____ 4 _____

3 BUILD IT! Friend Collector 👫

Build a project that asks the player for **3 friends' names** and stores them in a list called **friends**. Then make the cat say each name one by one.

- Use the **ask … and wait** block (Sensing) to get a name.
- Use **add (answer) to friends** to save it.
- Use a **repeat 3** loop with **item (i) of friends** to read them back.

☐ My cat remembers all 3 friends!

Tip — A **variable** holds *one* value. A **list** holds *many* values. Same idea, different sizes.

CHAPTER SIX

06

Conditionals (if / else)

Teach your sprite to think — to ask "is this true?" and choose what to do next, just like you do every day.

IF | IF / ELSE | COMPARISONS | QUIZ CHECKER

6.1 Yes or No · The Condition

Big Idea

A **condition** is a question that has only two answers — **YES** (true) or **NO** (false). Code uses conditions to **choose** what happens next.

You make condition-decisions every day

🌧️ Is it raining?

YES → take an umbrella.
NO → leave it at home.

🥛 Is the milk hot?

YES → wait 5 minutes.
NO → drink it now.

📚 Did I finish my homework?

YES → play.
NO → finish first!

In code, this looks like a fork in the road 🛣️

IF YES
Run the green blocks — go this way.

answer = 99 ?

IF NO
Run the red blocks — go that way instead.

The two control blocks you'll use

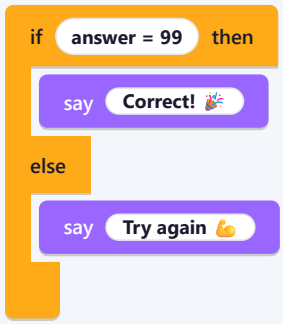
🔴 if < ... > then

RUN THE INSIDE BLOCKS ONLY WHEN ANSWER IS YES

Use this when there's **only something to do if YES** — and nothing to do if NO.

🔴 if < ... > then / else

TWO PATHS — ONE FOR YES, ONE FOR NO



Use this when **both** answers (YES and NO) need to do something.

6.2 Asking Yes/No · the green Operator blocks

Inside the diamond hole at the top of an *if* block, you slot a **green operator block**. That block asks the YES/NO question — and Scratch answers it for you.

The 3 most-used comparison blocks

= Equal to

`5 = 5`

YES if both sides are exactly the same.

< Less than

`3 < 7`

YES if the *left* number is smaller.

> Greater than

`9 > 2`

YES if the *left* number is bigger.

Mini-quiz · what's the answer?

Read each green block and circle YES or NO

`10 = 10`	YES	NO
`5 < 3`	YES	NO
`7 > 2`	YES	NO
`99 = 100`	YES	NO

Combining conditions

Sometimes one question isn't enough. You can **combine** two questions using:

`( ) and ( )`

`( ) or ( )`

`not ( )`

and = both must be YES. **or** = at least one is YES. **not** = flips YES↔NO.

Real-life example · "Can I go play?"

TWO CONDITIONS JOINED WITH "AND" — BOTH MUST BE YES

```
if homework_done = yes and time < 7 then
    say Yay! Time to play 🎮
else
    say Finish work first 📘
```

VEDIC CONNECT

Yāvadūnam — *"by the deficiency"*

Vedic Maths often asks: "How far is this number **from 100**?" — that's a comparison question, just like our **=** and **<** blocks.

→

IN SCRATCH

Compare → decide → act

The green operator does the comparing. The orange **if** block decides. Together they make your sprite *think*.

6.3 Mini-Project · Quiz Checker

What we're upgrading

We're going back to the Vedic Quiz from Chapter 5 and giving the cat a **brain**. After every question, the cat will *compare* the player's answer with the correct Vedic answer and say **"Right!"** or **"Try again"**.

The plan

1 Ask the question

Use the blue **ask** block. The player's reply lands in **answer** automatically.

2 Compare with the correct answer

Use the green **=** block: *answer = 99 ?*

3 Use if-else to give feedback

YES → say "Right!" + change score by 1. NO → say "Almost — the answer was 99".

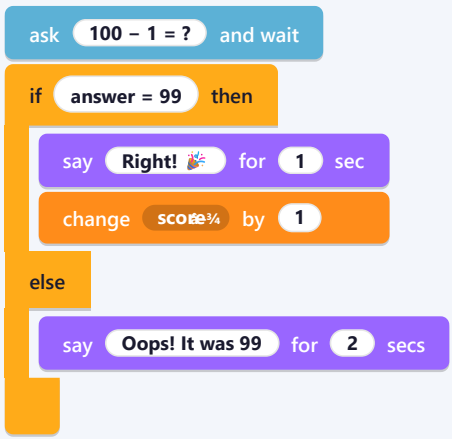
4 Repeat for all 5 questions

(Or use a list of correct answers — that's a Grade-5 trick!)

Sample question · 99 × 1 (using *Nikhilam* sutra)

The script for ONE question

ADD THIS INSIDE THE REPEAT-5 LOOP



What you'll see on the Stage

When the player answers 99, the cat cheers and the score box ticks up.

VEDIC SUTRA

Nikhilam Navataścaramaṁ Daśataḥ

"All from 9 and the last from 10." — the trick we used to compute **100 − 1 = 99** in one second, no borrowing

→

IN SCRATCH

The if-else is the judge

The cat doesn't know maths — it just **compares** the player's number with the correct one. Smart code, simple thinking.

needed.

YOUR TURN · ACTIVITY 6.A

Add 4 more checked questions

1. Copy the if-else above 4 more times — once for each of your 5 questions.
2. Change the question text and the correct answer in the green **=** block.
3. Test by purposely typing *wrong* answers — the cat should say "Oops!" 😾

- [ ] My quiz now scores by itself!

ðŸ¤– ASK A.I. **Stuck? Try this prompt —**

> âœ¨ In my Scratch quiz the cat asks **"What is 100 - 1?"**. I want it to say **"Right!"** if the player's **answer = 99**, and **"Oops, try again!"** otherwise. I'm using the **if … else** control block with the green **=** operator inside, but the **else** part never runs even when I type a wrong number. Can you check what might be wrong with my comparison?

💡 *Naming the **blocks you used** (if…else, green =) helps A.I. point straight at the bug.*

If... then... else!

Make the computer think. Decide. Choose. Just like you do every day.

1 TRUE OR FALSE? Tick the right box.

Condition	True	False
10 > 5	☐	☐
7 = 7	☐	☐
"cat" = "Cat"	☐	☐
100 < 99	☐	☐
(3 + 2) = 5	☐	☐

2 PREDICT What does the cat say?

The cat asks: "Pick a number." For each input below, write what the cat replies.

If you type...	Cat says...
15	
10	
3	
100	

3 BUILD IT! Pass / Fail Reporter

Build a script that asks the player for their **marks (out of 100)** and replies:

- If marks **≥ 90** → say **"Excellent!** 🌟**"**
- Else if marks **≥ 50** → say **"Pass** 😊**"**
- Else → say **"Try again** 💪**"**

How many **if ... else** blocks do you need (nested)?

_____

☐ I built a 3-level grader!

Tip — Conditions are just questions with **yes** or **no** answers. The cat acts only when the answer is **yes (true)**.

CHAPTER SEVEN

07

Operators & Maths Blocks

The green family — the maths brain of Scratch. We'll bring three famous Vedic sutras to life inside our code.

+ − × ÷ | JOIN & PICK | VEDIC ADDITION | VEDIC SUBTRACTION | NIKHILAM | URDHVA-TIRYAGBHYĀM

7.1 The Green Operators Family

Big Idea

Operators are the **maths and logic blocks**. They are shaped like **green pills** because they always *fit inside* other blocks — they don't run on their own, they help other blocks decide and calculate.

The maths operators (the four old friends)

➕ Addition

`5 + 3`

Gives **8**.

➖ Subtraction

`9 - 4`

Gives **5**.

✖ Multiplication

`6 × 7`

Gives **42**.

➗ Division

`12 / 4`

Gives **3**.

Two more very useful operators

🎲 pick random

`pick random 1 to 10`

Gives a surprise number every time. Perfect for quiz questions and dice.

🔗 join (glue text together)

`join Score: score`

Glues words and numbers into one sentence. Makes your sprite say smart things.

Operators slot *inside* other blocks

Notice how each green pill sits **inside** a white circle of another block. That's the magic of Scratch — small pieces fit together like LEGO!

3 WAYS OPERATORS ARE USED

```
set total¾ to 5 + 3
say join Total = total for 2 secs
move pick random 10 to 50 steps
```

ORDER MATTERS IN MATHS TOO!

Scratch follows the same rules you learn in school: **BODMAS** (Brackets → Of → Division → Multiplication → Addition → Subtraction). Use **nested green pills** to control the order, e.g. **(2 + 3) × 4 = 20**, but **2 + 3 × 4 = 14**.

7.2 Vedic Algorithm 1 · Vedic Addition

The Sutra

Kevalaih Saptakam Guṇyāt and **Śūnyaṁ Sāmyasamuccaye** — the ancient sages gave us *smarter ways* to add. Vedic addition means **fewer carries**, **faster mental sums**, and a chance to add **from left to right** — the way we naturally read numbers!

Vedic super-tricks for addition

TRICK

← Left-to-Right Addition

Instead of the usual right-to-left, add the **biggest place value first**. Adjust the number as you move right — whenever a column reaches 10+, bump the left digit up by 1. This is how mental maths champions do it!

Worked example — with the Dot Method

Add: **348 + 521 + 146**. Work column by column, right to left.

Dot Method · column by column — *Śuddha · "Pure column addition"*

ONES	8 + 1 + 6 = 15 → write **5**, place • above the tens column *(1 carry)*
TENS	4 + 2 + 4 = 10, plus the • makes 11 → write **1**, place • above the hundreds *(1 carry)*
HUNDREDS	3 + 5 + 1 = 9, plus the • makes 10 → write **10** *(the last column can spill into thousands)*
ANSWER	1 0 1 5 ✅ 348 + 521 + 146 = **1015**

Worked example — using Make Tens

Add: **7 + 3 + 8 + 2 + 5**. Spot the pairs that make 10 first!

```
  7
  3
  8
  2
+ 5
----
 25
```

Steps

1. **7 + 3 = 10** *(first pair)*
2. **8 + 2 = 10** *(second pair)*
3. Left over: **5**
4. **10 + 10 + 5 = 25** ✅

No carrying, no scratching — just pair, group and go!

WHY VEDIC ADDITION IS POWERFUL

You do **less writing**, use **fewer carries**, and can often reach the answer in your **head**. The same tricks scale up beautifully — whether you're adding three single digits or four 4-digit numbers. Ready to try?

Solve using your favourite Vedic trick!

Use **Make Tens**, the **Dot Method**, or **Left-to-Right**. Write the sum on the answer line under each stack.

1.

```
  4
  7
+ 5
---
```

2.

```
  8
  3
+ 6
---
```

3.

```
  9
  5
+ 2
---
```

4.

```
  6
  8
+ 4
---
```

5.

```
  7
  9
+ 3
---
```

6.

```
  5
  4
+ 8
---
```

7.

```
  3
  7
+ 9
---
```

8.

```
  2
  8
+ 6
---
```

9.

```
  9
  4
+ 5
---
```

10.

```
  8
  6
+ 7
---
```

11.

```
  348
  521
+ 146
-----
```

12.

```
  672
  184
+ 239
-----
```

13.

```
  405
  398
+ 507
-----
```

14.

```
  918
  276
+ 134
-----
```

15.

```
  731
  529
+ 168
-----
```

Tip — For single-digit stacks (Q1–10), first scan for pairs that make **10**. For 3-digit stacks (Q11–15), use the **Dot Method** — one column at a time!

Level up — big numbers, same tricks!

Now try 3-digit and 4-digit sums. The **Dot Method** is your best friend here — one column at a time, top to bottom.

16.

```
  482
  316
+ 295
-----
```

17.

```
  659
  207
+ 481
-----
```

18.

```
  874
  125
+ 699
-----
```

19.

```
  543
  612
+ 178
-----
```

20.

```
  296
  483
+ 721
-----
```

21.

```
 3482
 5917
 2604
+ 1489
------
```

22.

```
 7265
 1834
 5901
+ 274
------
```

23.

```
 9054
 2178
 3615
+ 4203
------
```

24.

```
 6429
 1857
  904
+ 7136
------
```

25.

```
 5298
 3467
 1859
+ 6274
------
```

26.

```
 8143
 2765
 3928
+ 1456
------
```

27.

```
 9731
 2684
 5419
+ 3207
------
```

28.

```
 4862
 7135
 2594
+ 4081
------
```

29.

```
 6509
 1846
 9723
+ 3158
------
```

30.

```
 4327
 5896
 2715
+ 6048
------
```

☐ I finished all 30 — the Vedic way! ⭐

7.3 Vedic Algorithm 2 · Vedic Subtraction

The Sutra

Nikhilam Navataścaramaṁ Daśataḥ — *"All from 9 and the last from 10."* This one sutra unlocks **super-fast subtraction**. With a little twist called the **Complement Trick**, we can subtract any number from any number — *without borrowing even once!*

Three Vedic super-tricks for subtraction

TRICK 1

🎯 Nikhilam · From 10, 100, 1000...

To find **1000 − 437**, apply the sutra: **9−4=5**, **9−3=6**, **10−7=3** → **563**. Every digit from 9, the last one from 10. That's it!

TRICK 2

🪄 The Complement Trick

For any subtraction, **round the bottom number up** to the nearest 10, 100 or 1000. Add the tiny **complement** to the top, then subtract the round number. No borrowing, no scratching!

TRICK 3

🧠 Think "What Completes the Top?"

For small stacks like **15 − 7**, don't take away — *add up*! Ask: **7 + ? = 15**. Answer **8**. Your brain is already fast at addition; use that speed for subtraction too.

Worked example — the Complement Trick

Subtract: **452 − 76 = ?** The bottom (76) is close to **100** — let's use that.

Complement Method · step by step — *Nikhilam · complement to 100*

STEP 1	Round the bottom (76) up to the nearest 100 → **100**
STEP 2	Find the **complement** of 76 using Nikhilam: **9−7 = 2**, **10−6 = 4** → **24**
STEP 3	**Add** the complement to the top: **452 + 24 = 476**
STEP 4	**Subtract** the round number: **476 − 100 = 376**
ANSWER	**376** ✅ 452 − 76 = **376** *(no borrowing!)*

Bigger example — 5604 − 2786

Same trick, bigger numbers. Round **2786** up to **3000**.

```
  5604
− 2786
------
  2818
```

Steps

1. Round **2786** up to **3000**.
2. Complement = **3000 − 2786 = 214**.
3. Add complement to top: **5604 + 214 = 5818**.
4. Subtract the round number: **5818 − 3000 = 2818** ✅

The bigger the numbers, the more time the trick saves!

WHY VEDIC SUBTRACTION FEELS LIKE MAGIC

Regular subtraction makes you **borrow** across columns — messy and slow. The **complement trick** turns subtraction into *a small addition + a subtraction from a round number*. Both parts are easy. Try it on the worksheet — you'll never subtract the old way again!

Warm-up — small numbers, big brains!

Start with the "**What completes the top?**" trick. From Q11 onwards, try the **Complement Trick** — round the bottom up.

1.

  9
− 4

2.

  15
− 7

3.

  18
− 9

4.

  27
− 8

5.

  34
− 6

6.

  42
− 9

7.

  55
− 7

8.

  63
− 8

9.

  71
− 9

10.

  88
− 5

11.

  124
− 8

12.

  136
− 19

13.

  205
− 17

14.

  248
− 39

15.

  317
− 28

Tip — For Q11–15, round the bottom up to the nearest **10**. Example: **136 − 19**. Round 19 → 20, complement = 1. So **136 + 1 − 20 = 137 − 20 = 117**. ✨

Level up — the Complement Trick shines!

For each sum: round the **bottom** up to the nearest **100** or **1000**, add the tiny complement to the top, then subtract the round number.

16.

  452
− 76

17.

  584
− 98

18.

  673
− 157

19.

  805
− 286

20.

  946
− 458

21.

  1234
− 567

22.

  2456
− 789

23.

  3810
− 1945

24.

  4728
− 1859

25.

  5604
− 2786

26.

  6842
− 3957

27.

  7420
− 1865

28.

  8531
− 4978

29.

  9164
− 5879

30.

  12035
− 6847

☐ I finished all 30 subtractions — the Vedic way, no borrowing! ⭐

7.4 Vedic Algorithm 3 · Nikhilam in Scratch

The Sutra

Nikhilam Navataścaramaṁ Daśataḥ — *"All from 9 and the last from 10."* A magical way to subtract from 100, 1000, 10000 — without borrowing!

How a human does it (mentally)

Goal: **1000 − 437 = ?**

Subtract from	1000	*Nikhilam · "All from 9, last from 10"*
STEP 1	4 → 9 − 4 = 5	*(all from 9)*
STEP 2	3 → 9 − 3 = 6	*(all from 9)*
STEP 3	7 → 10 − 7 = 3	*(last from 10)*
ANSWER	5 6 3 ☑ 1000 − 437 = **563**	

Now let's teach Scratch to do it

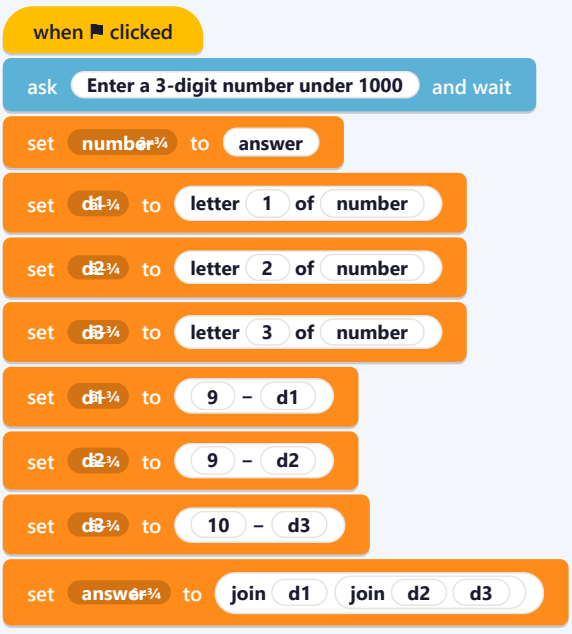
We'll keep it simple — for any 3-digit number, our cat will **read each digit**, apply the sutra, and show the answer.

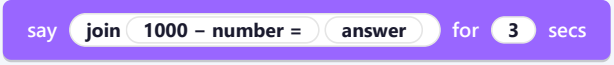
Variables we need

number 437 | d1 4 | d2 3 | d3 7 | answer 563

The script

STEP-BY-STEP NIKHILAM IN SCRATCH (FOR 1000 − NUMBER)





WHY IT WORKS

The **letter N of** operator (from the green family) lets us peek at one digit of a number — exactly what the sutra asks us to do, one digit at a time. We then **join** the three results to get the final answer.

YOUR TURN · ACTIVITY 7.A

Try it with three different numbers

Run your project and write your own results below.

Input	Cat said...	My check
1000 − 248		
1000 − 591		
1000 − 803		

- [ ] My Scratch sprite knows Nikhilam!

7.5 Vedic Algorithm 4 · Urdhva Multiplication

The Sutra

Urdhva-Tiryagbhyām — "*Vertically and Crosswise.*" The most famous Vedic shortcut for multiplying two numbers in one line.

How a human does it (for two 2-digit numbers)

Goal: **23 × 12 = ?** Let's call it **(a b) × (c d)** where **a=2, b=3, c=1, d=2**.

Multiply	23 × 12	*Urdhva · "Vertically & Crosswise"*
VERTICAL (RIGHT)	b × d = 3 × 2 = 6	→ ones place
CROSSWISE	(a × d) + (b × c) = (2×2) + (3×1) = 7	→ tens place
VERTICAL (LEFT)	a × c = 2 × 1 = 2	→ hundreds place
ANSWER	2 7 6 ☑ 23 × 12 = **276**	*(carry handled if needed)*

In Scratch — the cat becomes a Vedic calculator

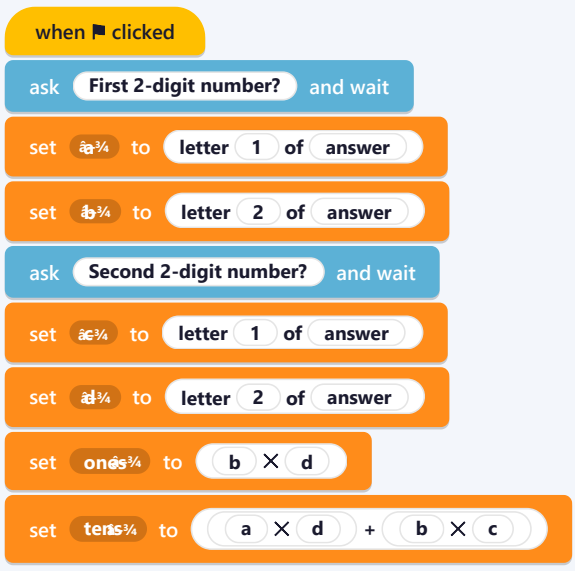
Variables we need

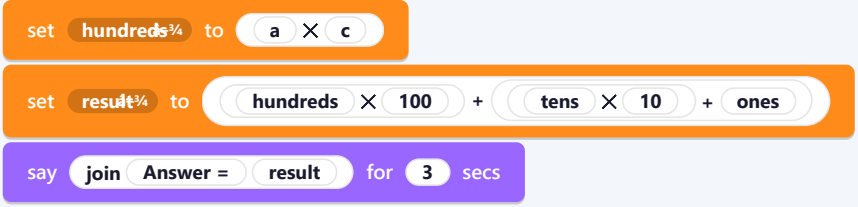
a 2 | b 3 | c 1 | d 2 | ones 6 | tens 7 | hundreds 2

result 276

The script

READS TWO 2-DIGIT NUMBERS AND USES THE SUTRA TO MULTIPLY THEM





WHY THIS ROCKS

One line · One sutra · One answer

The same sutra works for 3-digit × 3-digit, 4-digit × 4-digit, and beyond — just more crosswise pairs!

→

IN SCRATCH

Maths sutra → Maths code

You just translated an ancient sutra into a working program. **That**, dear coder, is real magic. ✨

YOUR TURN · ACTIVITY 7.B

Vedic vs. Calculator Race

1. Run your Scratch project and ask it: **23 × 12**, **14 × 21**, **32 × 13**.
2. Solve the same questions *mentally* using the sutra.
3. Time yourself with a stopwatch — who's faster, you or Scratch? 🏁

☐ I built a Vedic multiplier in Scratch!

ðŸ¤–

ASK A.I. **Stuck? Try this prompt —**

```
âœ️ I'm using the Urdhva-Tiryagbhyam (vertically & crosswise) sutra in Scratch to multiply two 2-digit numbers like 23 × 12. I have variables a, b, c, d, ones, tens, hundreds, result. I'm stuck on the green operator blocks for tens, which should equal (a × d) + (b × c). How do I nest the green × and + blocks correctly inside one set tens to block?
```

💡 *The word **"nest"** tells A.I. you mean blocks-inside-blocks — super useful for the green Operators family.*

7.6 Multiplication Warm-up 2-digit × 1-digit

Sharpen your multiplication muscle!

Warm up with 2-digit × 1-digit sums *before* we unlock the Vedic shortcuts on the next pages. Use **Urdhva-Tiryagbhyam** (vertically & crosswise) or the classic column method.

How to solve — the Vedic way (Urdhva applied to 2 × 1 digits)

Take a 2-digit number **(a b)** and a 1-digit number **c**. The answer is: **(a × c)** carried on to **(b × c)**.

78 × 6 → 7×6 = 42 · 8×6 = 48
→ 42 <shift> 48 = 420 + 48 = **468**

36 × 5 → 3×5 = 15 · 6×5 = 30
→ 150 + 30 = **180**

43 × 9 → 4×9 = 36 · 3×9 = 27
→ 360 + 27 = **387**

Exercise · Solve mentally, then write the answer

1. 78 × 6 = _____
2. 42 × 9 = _____
3. 19 × 8 = _____
4. 20 × 8 = _____
5. 36 × 5 = _____
6. 83 × 6 = _____
7. 54 × 8 = _____
8. 95 × 7 = _____
9. 43 × 9 = _____
10. 76 × 4 = _____
11. 62 × 7 = _____
12. 48 × 3 = _____

Tip — Try solving each one *without* writing intermediate steps. Vedic maths is about doing it in **one line, in your head**. Speed comes with practice — race yourself with a stopwatch!

The "sum of neighbours" trick

Multiplying *any* number by **11** takes just one step in Vedic maths — no long multiplication needed. Learn the method, then blaze through the practice sums.

Describe the Method — × *11 in one line*

2-digit A B × 11: write **A (A + B) B**. · **3-digit A B C × 11:** write **A (A + B) (B + C) C**. · **4-digit A B C D × 11:** write **A (A + B) (B + C) (C + D) D**. If any sum is **≥ 10**, keep the ones digit and *carry 1* to the left.

27 × 11 → 2 _ 7 with (2+7)=9
Answer = **297**

78 × 11 → 7 _ 8 with (7+8)=15 → carry
(7+1) _ 5 _ 8 = **858**

134 × 11 → 1 (1+3) (3+4) 4
= **1474**

2843 × 11 → carry chain
= **31273**

1.

27
× 11

2.

34
× 11

3.

52
× 11

4.

96
× 11

5.

38
× 11

6.

69
× 11

7.

78
× 11

8.

83
× 11

9.

134
× 11

10.

205
× 11

11.

439
× 11

12.

516
× 11

13.

704
× 11

14.

659
× 11

15.

783
× 11

16.

801
× 11

17.

368
× 11

18.

736
× 11

19.

$$\begin{array}{r} 254 \\ \times\ 11 \\ \hline \\ \hline \end{array}$$

20.

$$\begin{array}{r} 327 \\ \times\ 11 \\ \hline \\ \hline \end{array}$$

Tip — Q1–8 use the 2-digit rule. Q9–20 use the 3-digit rule. Watch for carries when neighbour sums are 10 or more!

7.7 Multiply by 11 Part 2 · Q21–40

Bigger numbers, same trick!

Quick recap: copy the outer digits and put the **sum of every neighbouring pair** in between. Carry 1 to the left whenever a sum is ≥ 10. Ready — set — solve!

21.

5 8 4
× 1 1

22.

9 3 7
× 1 1

23.

5 4 9
× 1 1

24.

6 2 7
× 1 1

25.

1 7 2 5
× 1 1

26.

2 8 4 3
× 1 1

27.

5 2 1 9
× 1 1

28.

6 0 5 4
× 1 1

29.

3 2 4 7
× 1 1

30.

7 8 2 3
× 1 1

31.

8 5 0 9
× 1 1

32.

2 5 6 4
× 1 1

33.

3 0 1 2
× 1 1

34.

1 5 4 6
× 1 1

35.

4 1 5 7
× 1 1

36.

3 2 5 8
× 1 1

37.

4 1 2 0
× 1 1

38.

5 2 7 4
× 1 1

39.

7 2 1 8
× 1 1

40.

3 6 0 8
× 1 1

Speed tip — With 4-digit numbers you get **three** neighbour-sums (between 4 digits). Work *right-to-left* so carries flow naturally into the next digit.

7.8 Multiply by 9, 99, 999 … (Nikhilam)

Multiplication with equal digits of "9"

When the multiplier is made *entirely* of 9s (like 9, 99, 999, 9999), the answer appears almost by magic — using the sutra **Nikhilam Navataścaramaṁ Daśataḥ** ("all from 9, last from 10").

Describe the Method — *N* × *(row of 9s of the same length)*

Case 1 — same number of digits. For an *n*-digit number **N** multiplied by **n nines**:

1. **Left half** of the answer = **N − 1**
2. **Right half** of the answer = complement of **N** (i.e. **10^n − N**) — "all from 9, last from 10"
3. Join the two halves. Done!

88 × 99
Left = 88 − 1 = **87**
Right = 100 − 88 = **12**
Answer = **8712**

564 × 999
Left = 564 − 1 = **563**
Right = 1000 − 564 = **436**
Answer = **563436**

7 × 9
Left = 7 − 1 = **6**
Right = 10 − 7 = **3**
Answer = **63**

Handy shortcut for "all from 9, last from 10": subtract every digit from **9**, except the **last** digit, which you subtract from **10**. Example: complement of 564 = (9−5)(9−6)(10−4) = 4 3 6 = **436**. ✅

1.

74 × 99 = _____

2.

82 × 99 = _____

3.

35 × 99 = _____

4.

68 × 99 = _____

5.

91 × 99 = _____

6.

62 × 99 = _____

7.

95 × 99 = _____

8.

38 × 99 = _____

9.

47 × 99 = _____

10.

18 × 99 = _____

11.

45 × 99 = _____

12.

89 × 99 = _____

13.

63 × 99 = _____

14.

77 × 99 = _____

15.

31 × 99 = _____

16.

$$\begin{array}{r} 58 \\ \times\ 99 \\ \hline \\ \hline \end{array}$$

17.

$$\begin{array}{r} 87 \\ \times\ 99 \\ \hline \\ \hline \end{array}$$

18.

$$\begin{array}{r} 54 \\ \times\ 99 \\ \hline \\ \hline \end{array}$$

19.

$$\begin{array}{r} 43 \\ \times\ 99 \\ \hline \\ \hline \end{array}$$

20.

$$\begin{array}{r} 96 \\ \times\ 99 \\ \hline \\ \hline \end{array}$$

21.

$$\begin{array}{r} 40 \\ \times\ 99 \\ \hline \\ \hline \end{array}$$

22.

$$\begin{array}{r} 37 \\ \times\ 99 \\ \hline \\ \hline \end{array}$$

23.

$$\begin{array}{r} 52 \\ \times\ 99 \\ \hline \\ \hline \end{array}$$

24.

$$\begin{array}{r} 70 \\ \times\ 99 \\ \hline \\ \hline \end{array}$$

Self-check tip — The answer of any 2-digit × 99 always has **4 digits**, and the **sum of all four digits is a multiple of 9**. Try it — 74 × 99 = 7326 → 7+3+2+6 = 18. 🎯

7.8 Multiply by 999 & 9999 (Nikhilam · continued)

Same trick — bigger numbers!

The Nikhilam sutra scales up beautifully. 3-digit × 999 and 4-digit × 9999 use exactly the same "left = N − 1, right = complement" rule.

Quick recap — *equal digits of 9*

Left half = **N − 1** · **Right half** = complement of **N** (all digits from 9, last digit from 10). Right half must have the **same number of digits** as N — pad with leading zeros if needed.

359 × 999

Left = 358 · Right = (9−3)(9−5)(10−9) = **641**

Answer = **358641**

5346 × 9999

Left = 5345 · Right = (9−5)(9−3)(9−4)(10−6) = **4654**

Answer = **53454654**

What if the multiplier has *more* 9s than N?

Example 191 × 999 uses same rule (both are 3-digit vs 999). But **773 × 999** works too: 772 | 227 = **772227**. Just check digit counts match!

Special case — Q28 (674 × 999) and **Q30 (999 × 989)**: In Q30 the multiplier 989 is *not* a row of 9s, so use classic column multiplication or Urdhva. Watch the "odd one out" — great puzzle!

25.

359
× 999

26.

432
× 999

27.

400
× 999

28.

674
× 999

29.

191
× 999

30.

999
× 989

31.

773
× 999

32.

858
× 999

33.

5346
× 9999

34.

1827
× 9999

35.

5083
× 9999

36.

9999
× 9435

37.

2850
× 9999

38.

3948
× 9999

39.

5714
× 9999

40.

$$\begin{array}{r} 7070 \\ \times\ 9999 \\ \hline \\ \hline \end{array}$$

Race your Scratch cat! — Build a tiny Scratch project that asks for a number and multiplies it by 999 using the sutra. Can you beat it with mental maths? Time yourself for extra fun. ⏱️

Turn tricky division into easy multiplication!

Dividing by **5**, **25** or **50** looks scary — but each one is secretly a × in disguise. Learn one small rule and blaze through the practice.

Describe the Method — *Divide by "half of 10, 100..."*

The Vedic idea: any divisor that is a **friendly fraction of 10 or 100** can be flipped into a multiplication. Because **5 = 10 ÷ 2**, **25 = 100 ÷ 4**, and **50 = 100 ÷ 2**, we get three tiny rules:

1. **÷ 5** = **× 2**, then move the decimal **one** place left *(same as ÷ 10)*
2. **÷ 25** = **× 4**, then move the decimal **two** places left *(same as ÷ 100)*
3. **÷ 50** = **× 2**, then move the decimal **two** places left *(same as ÷ 100)*

35 ÷ 5	39 ÷ 5	75 ÷ 25	250 ÷ 50
35 × 2 = 70	39 × 2 = 78	75 × 4 = 300	250 × 2 = 500
70 ÷ 10 = **7**	78 ÷ 10 = **7.8**	300 ÷ 100 = **3**	500 ÷ 100 = **5**

Tip — If the answer has a decimal (like 39 ÷ 5 = 7.8), you can also read it as a remainder: **7 remainder 4**. Both are correct — pick the form your teacher expects.

1.

35 ÷ 5 =

2.

20 ÷ 5 =

3.

45 ÷ 5 =

4.

80 ÷ 5 =

5.

95 ÷ 5 =

6.

70 ÷ 5 =

7.

125 ÷ 5 =

8.

190 ÷ 5 =

9.

39 ÷ 5 =

10.

76 ÷ 5 =

11.

83 ÷ 5 =

12.

56 ÷ 5 =

13.

91 ÷ 5 =

14.

49 ÷ 5 =

15.

87 ÷ 5 =

16.

66 ÷ 5 = _____

_____

17.

75 ÷ 25 = _____

_____

18.

150 ÷ 25 = _____

_____

19.

225 ÷ 25 = _____

_____

20.

375 ÷ 25 = _____

_____

Tip — Q1–16 use the **÷ 5** rule (× 2, then ÷ 10). Q17–20 switch to the **÷ 25** rule (× 4, then ÷ 100). Watch out for decimals when the number isn't a clean multiple!

7.9 Divide by 25 & 50 Part 2 · Q21–40

Bigger numbers, same magic!

Quick recap — ÷ 25 = × 4 then ÷ 100. ÷ 50 = × 2 then ÷ 100. Move the decimal *two* places for both. Ready — set — divide!

21. $850 \div 25 =$

22. $925 \div 25 =$

23. $1175 \div 25 =$

24. $1650 \div 25 =$

25. $217 \div 25 =$

26. $364 \div 25 =$

27. $510 \div 25 =$

28. $321 \div 25 =$

29. $187 \div 25 =$

30. $462 \div 25 =$

31. $531 \div 25 =$

32. $278 \div 25 =$

33. $250 \div 50 =$

34. $400 \div 50 =$

35. $650 \div 50 =$

36. $950 \div 50 =$

37. $1550 \div 50 =$

38. $1900 \div 50 =$

39. $2350 \div 50 =$

40. $5407 \div 50 =$

Self-check tip — Any number that ends in **0, 25, 50 or 75** divides *cleanly* by 25. Any number ending in **0** or **50** divides cleanly by 50. If your answer has a decimal, you did the maths right — the number simply isn't a perfect multiple!

Crunch the numbers!

The green family is your maths toolbox. Time to wield it like a pro.

1 CALCULATE Solve mentally — then check in Scratch.

Given: **a = 2, b = 3, c = 4, d = 5**. Find the value of:

Expression	Working	Answer
(a + b)		
(c × d)		
(a × d) + (b × c)		
(a + b) × (c + d)		

2 BLOCK EXPRESSION Draw the green block stack.

How would you write **(5 × 2) + 3** using Scratch operator blocks? Sketch the block in the box (or describe which blocks go inside which).

Hint: nest a green × block *inside* a green + block.

3 BUILD IT! Vedic Calculator — Squares ending in 5

The sutra **Ekadhikena Purvena** says: **$(N5)^2 = N \times (N+1)$** followed by **25**. Example: **$35^2 = 3 \times 4 = 12 \Rightarrow 1225$**.

1. Build a Scratch script that asks for a number **N** (like 3, 4, 5…).
2. Compute **front = N × (N + 1)**.
3. Use **join** to display **front** followed by **25**.

Write the answers your project should give for these inputs:

Input N	front (N×(N+1))	Answer (join with 25)
3 (so 35^2)		
7 (so 75^2)		
9 (so 95^2)		

☐ My Vedic Squarer works!

Tip — Inside Scratch, big maths is just *small green blocks stacked inside each other*. Build slowly, test each step.

FOR SELF-CHECK, PARENTS & TEACHERS

✓

Answer Key

Solutions to every exercise from Chapters 3 to 7. Try the questions first on your own — then peek here to check your work. The struggle is where the real learning happens!

CH 3 · SEQUENCING

CH 4 · LOOPS

CH 5 · VARIABLES

CH 6 · CONDITIONALS

CH 7 · OPERATORS

Sequencing & Events — Solutions

Try the questions first. *Then* peek here to check.

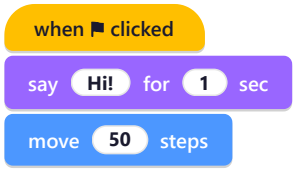
1 PREDICT Order on the stage

Blocks always run **top to bottom**. So the order is: **move 30 → say "Hello!" → play meow → turn 15°.**

Order	What happens
1	Cat moves 30 steps
2	Cat says "Hello!"
3	Cat plays meow sound
4	Cat turns 15°

2 FIX THE BUG Correct stack

Riya wants *say* "*Hi*" first, *then jump*. Correct order:



3 BUILD IT! Birthday Sprite

The Events block that starts your script when the space-bar is pressed:

when [space] key pressed (yellow Events block, with the dropdown set to *space*)

Remember — the block at the **top of the stack** always runs *first*. Change the order, change the story.

Loops — Solutions

One loop block can do the work of dozens. Check your counts below.

1 COUNT IT How many times?

Loop	Inside runs how many times?
repeat 10	**10** times
repeat 25	**25** times
forever	**Forever** — until you press the red ■ (stop)
repeat 4 inside repeat 3	**12** times (outer 3 × inner 4)

2 MATCH THE SHAPE repeat × turn = ?

Rule: turn = 360° ÷ number of sides.

Shape	repeat	turn (degrees)
Triangle	3	**120°**
Square	**4**	90°
Pentagon	5	**72°**
Octagon	**8**	**45°**

3 BUILD IT! Rocket Countdown

To make the counter go down from 10 to 1, use:

change counter by −1 (negative one — orange Variables block)

Remember — a loop is just "*do this again*". If you're copying blocks more than twice — switch to a loop.

Variables & Lists — Solutions

Trace each step in order. Remember: **set** replaces, **change** adds to.

1 TRACE IT What's in the score box?

Start: **score = 0**.

Instruction	score is now...
change score by 5	**5**
change score by 3	**8**
set score to 10	**10** (set replaces!)
change score by −4	**6**
change score by 1	**7**

2 LIST TRACE Final list

Trace: [apple] → [apple, banana] → [apple, banana, cherry] → **delete item 1** → [banana, cherry] → [banana, cherry, date].

Final **fruits** list (only 3 items remain):

1	2	3	4
banana	**cherry**	**date**	— empty —

3 BUILD IT! Friend Collector

Outline of a correct project:

1. Make a list called **friends**.
2. Use **repeat 3** with: *ask "Friend's name?" and wait*, then *add (answer) to friends*.
3. Use another **repeat 3** with a counter **i**: *say (item i of friends)* for 1 sec, then *change i by 1*.

Remember — deleting item 1 always shifts every other item *up* by one. That's why the final list is only 3 long, not 4.

Conditionals — Solutions

Every condition is a question with a **yes/no** answer. Let's check yours.

1 TRUE OR FALSE? Answers

Condition	Answer
10 > 5	**True**
7 = 7	**True**
"cat" = "Cat"	**True** in Scratch (Scratch ignores capital letters in =). In most other coding languages this would be **False**.
100 < 99	**False**
(3 + 2) = 5	**True**

2 PREDICT What does the cat say?

Condition: **answer > 10**. If true → "Big!", else → "Small!".

If you type...	Cat says...
15	**Big!** (15 > 10 is true)
10	**Small!** (10 > 10 is false — > means *strictly* greater)
3	**Small!**
100	**Big!**

3 BUILD IT! Pass / Fail Reporter

You need **2 nested if-else blocks**:

1. Outer: **if** answer ≥ 90 → say "Excellent! 🌟" **else** ...
2. Inner (in the else): **if** answer ≥ 50 → say "Pass 😊" **else** say "Try again 💪".

Remember — the symbol **>** is *strictly greater than*. For "10 or more", use **≥** (or check **not (answer < 10)**).

Operators & Maths — Solutions

BODMAS still rules — even when you're stacking green blocks.

1 CALCULATE a = 2, b = 3, c = 4, d = 5

Expression	Working	Answer
(a + b)	2 + 3	**5**
(c × d)	4 × 5	**20**
(a × d) + (b × c)	(2×5) + (3×4) = 10 + 12	**22**
(a + b) × (c + d)	(2+3) × (4+5) = 5 × 9	**45**

2 BLOCK EXPRESSION (5 × 2) + 3

Nest a green **×** **block** *inside* a green **+** **block**:

```
( 5 × 2  + 3 )
```

The final answer is **(5 × 2) + 3 = 10 + 3 = 13**.

3 BUILD IT! Vedic Calculator — Squares ending in 5

Rule: **front = N × (N + 1)**, then **join** with **25**.

Input N	front (N × (N+1))	Answer (join with 25)
3 (35^2)	3 × 4 = **12**	**1225**
7 (75^2)	7 × 8 = **56**	**5625**
9 (95^2)	9 × 10 = **90**	**9025**

Remember — *Ekadhikena Purvena* means **"by one more than the previous one"**. For 35^2, the previous digit is 3, so we multiply 3 by **one more** = 4. Then stick 25 on the end. Magic!

BONUS CAPSTONE · TAKE-HOME PROJECT

The *Water Cycle* Animation

*You have learned events, loops, variables, conditionals, operators and Vedic sutras. Now mix them all together to build a real **Science animation** — the journey of a single water drop from the sea to the sky and back as rain.*

5 SPRITES · BROADCASTS · CLONING 🆕 · VARIABLES · REAL SCIENCE · SHOW AT HOME 🏠

What we're building

An animated Science scene that shows the **three big stages** of the water cycle, looping forever:

1. Evaporation — the Sun heats the sea, a tiny water drop rises into the sky. **2. Condensation** — many drops meet and the cloud grows bigger. **3. Precipitation** — when the cloud is heavy enough, it rains.

Meet your 5 sprites + 1 boss

🎬 Stage (the boss)

Doesn't move — but **tells everyone what to do**. Owns the master loop and two variables: **Cycle** and **drops**.

🌞 Sun

Just pulses gently in the top-right corner. Decorative — no broadcasts from here.

💧 WaterDrop

Hides at the bottom. When it hears "evaporate", it shows up, says "*Evaporation!*" and glides up to the cloud.

☁️ Cloud

Sits at the top. Grows bigger every "condense". Resets to its small size after every rainstorm.

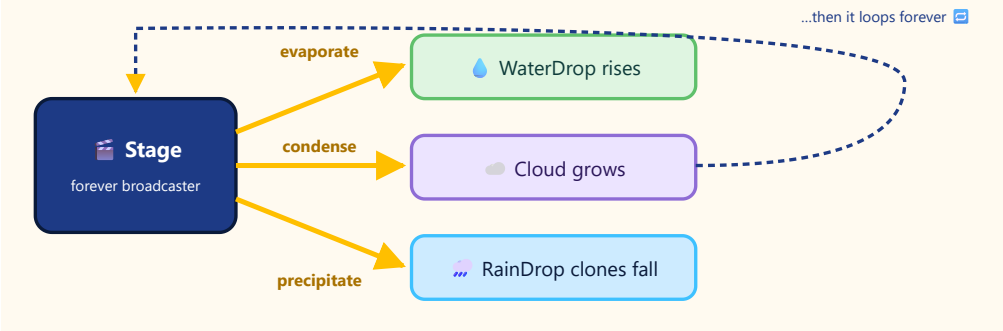
🌧️ RainDrop

Hides on flag. When it hears "precipitate", it **clones itself 8 times** and the clones fall to the ground.

📦 Variables

Cycle — how many full water cycles we've finished.
drops — how many drops have collected in the cloud this round.

How the sprites talk — the broadcast flow 📣



The Stage is the only sprite that broadcasts. WaterDrop, Cloud and RainDrop just **listen** and react.

Step 1 · Paint the backdrop 🎨

Click the **Stage** (right side) → **Backdrops** tab → **Paint**. Make a sky that fades from blue at the top to sand at the bottom, with a dark blue sea at the bottom. This is the world your animation lives in.

Step 2 · The Sun — gentle pulse 🌞

Add a Sun sprite (paint a yellow circle). Place it at **x = 180, y = 130**. The Sun just **looks** alive — it doesn't control the cycle.

SUN SPRITE

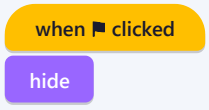
💡 **Why not let the Sun broadcast?**
If the Sun shouted **evaporate!** every half-second, it would *restart* the WaterDrop's script before the drop had time to glide. The Sun must stay **decorative**. The full secret is on the next page.

Step 3 · The WaterDrop — rises on cue 💧

Paint a small blue tear-drop sprite. It needs **two scripts**: hide at the start, and rise when it hears "*evaporate*".

Script A · Start hidden

WATERDROP SPRITE



Script B · When I hear "evaporate"

WATERDROP SPRITE · RECEIVES BROADCAST

REAL SCIENCE
Evaporation

→

IN SCRATCH
Glide = animation

Heat from the Sun gives water molecules so much energy that they **turn into vapour** and rise into the sky. Our drop's upward glide shows exactly that.

The **glide** block moves the sprite smoothly from one (x, y) to another in a chosen time. Perfect for showing motion in nature.

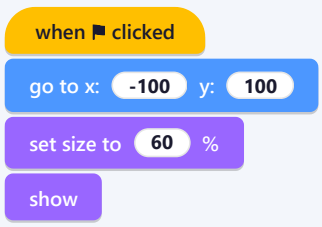
Step 4–5 · Cloud & RainDrop

Step 4 · The Cloud — three short scripts

Paint a fluffy white cloud. Place it at **x = -100, y = 100**. The Cloud listens to **two** broadcasts and resets itself with one start-up script.

A · Set up

CLOUD · ON FLAG

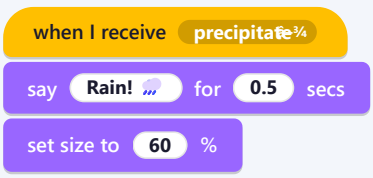


B · On "condense"

CLOUD GROWS

C · On "precipitate"

CLOUD RESETS

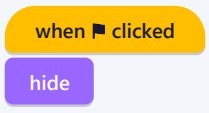


Step 5 · The RainDrop — meet cloning

A **clone** is a copy of a sprite that Scratch creates while the project is running. Perfect for rain: one tiny RainDrop sprite can become *eight* falling drops.

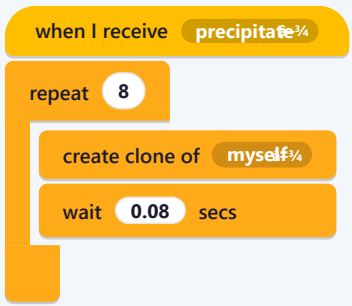
A · Hide at start

RAINDROP · MAIN



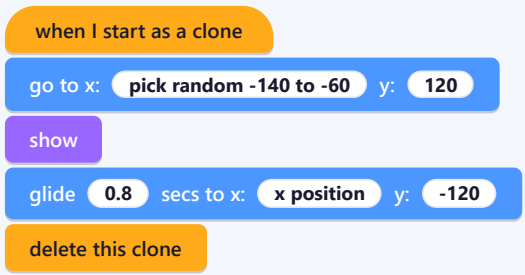
B · Spawn 8 clones

ON "PRECIPITATE"



C · Each clone falls

CLONE BEHAVIOUR



> 🧠 **Why delete this clone?**
> Without it, every cycle would leave 8 more drops frozen at the bottom of the stage. After hundreds of cycles your Scratch project would slow down. **Delete this clone** = good housekeeping. Always clean up your clones!

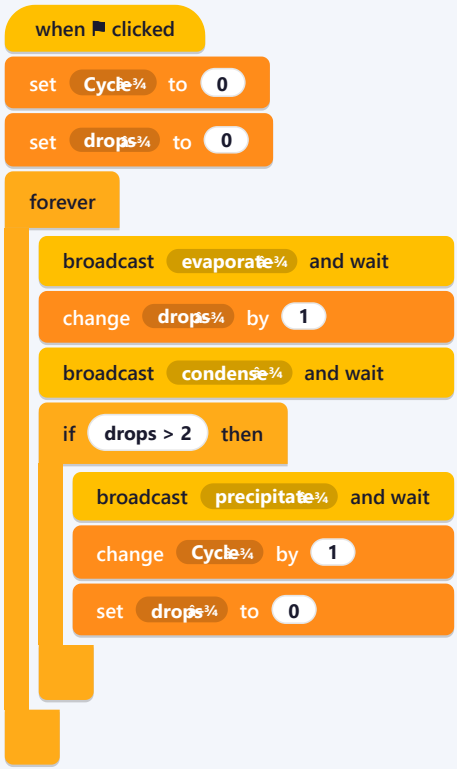
Step 6 · The Stage Drives the Cycle

BONUS
TAKE-HOME PROJECT

Click on the **Stage** (right side, below the green flag). The Stage gets its *own* script area. Make **two variables** *for this sprite only*: **Cycle** (starts at 0) and **drops** (starts at 0). These belong to the whole project.

The master loop 🔁

STAGE · THE CONDUCTOR OF THE ORCHESTRA



⚠️ The "broadcast and wait" trap

The bug we have to avoid
If you use the plain **broadcast** block in a fast **forever** loop, Scratch will fire *evaporate* again **before** the WaterDrop finishes gliding — and Scratch **restarts** the receiving script from the top. Result: you only ever see "Evaporation!" — the drop never reaches the cloud. 😱

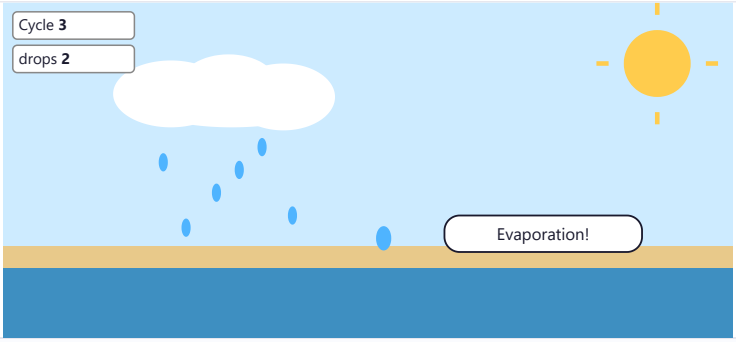
The fix — use **broadcast … and wait**. The Stage now *pauses* until the WaterDrop has finished its glide, then fires the next message. Each phase gets to finish. Beautiful!

Timeline — what happens in one cycle ⏱️

Time	Who acts	What you see
0.0 s	Stage → "evaporate"	WaterDrop appears at sea, says *"Evaporation!"*
0.5–2.0 s	WaterDrop	Glides up to the cloud, hides
2.0 s	Stage	**drops** changes from 0 → 1 → 2 → 3 (one per cycle)

2.0 s	Stage → "condense"	Cloud says "*Condensation!*" and grows by 15%
~3.0 s	Stage checks **drops > 2**	After the 3rd drop, the condition is true → time to rain!
~3.5 s	Stage → "precipitate"	8 RainDrop clones fall, Cloud resets to small, **Cycle** goes up by 1
loop	Stage	Back to "evaporate" — the water cycle continues 🔁

What you should see on the stage 🎬



Cycle = 3 means the water has already gone sea → sky → rain → sea **three times**. drops = 2 means one more evaporation will trigger the next rainfall.

⭐ Bonus challenges — try at least one!

🌈 Add a rainbow

After "precipitate", broadcast **rainbow** — and let a new Rainbow sprite **show** for 2 seconds, then **hide**.

❄️ Make it snow

Add a Temperature variable. When Temperature < 0, broadcast **snow** instead of precipitate. Use a white snowflake costume.

🔊 Add sound effects

From the **Sounds** tab, add a "drip" sound to RainDrop and play it when each clone starts.

⛰️ Add a mountain

Paint a brown mountain on the backdrop. Where do **rivers** fit into the water cycle?

YOUR TURN · ACTIVITY B.1

Plan, then build

1. Open Scratch (online: **scratch.mit.edu** · offline: Scratch Desktop). Start a new project named **"Water Cycle".**
2. Make all 5 sprites + 2 Stage variables (**Cycle** and **drops**).
3. Type the scripts from pages B2 → B4. Test by clicking the green flag.
4. Bonus 🌟: Pick **one** challenge from above and add it. Tick it here once it works:

- [ ] 🌈 Rainbow
- [ ] ❄️ Snow
- [ ] 🔊 Sound
- [ ] ⛰️ Mountain
- [ ] I showed my finished project to a parent or friend!

ðŸ¤– ASK A.I. **Stuck? Try this prompt —**

> âœï¸ In Scratch, my **Stage** uses a **forever** loop with **broadcast evaporate and wait** → **broadcast condense and wait** → if **drops > 2** then **broadcast precipitate and wait.** My **WaterDrop** hears "evaporate", says *"Evaporation!"* and glides up — but the cloud never grows. What might I have done wrong, and what should I check first?

💡 *Telling A.I. the **order of broadcasts** and which sprite listens to which one helps it find the bug fast.*

WELL DONE, CODER!

You finished *Volume 1*

*From your first sprite to your own Vedic calculator — every block you snapped, every sutra you used, every bug you fixed has made you a stronger thinker. See you in **Volume 2 · Builders** for Grade 5!*

SEQUENCING ✓ LOOPS ✓ VARIABLES & LISTS ✓ CONDITIONALS ✓ OPERATORS ✓ VEDIC SUTRAS ✓ SELF-PRACTICE ✓